

Fraus

Medição de satisfação sem LLM em runtime

Table of contents

1. Fraus	6
1.1 O que ler primeiro	6
1.2 Sem LLM em runtime	6
1.3 Baixar	6
2. A tese	7
2.1 O problema	7
2.2 A aposta	7
2.3 O que sai disso, e que nada mais consegue calcular	7
2.4 O que a tese NÃO promete	7
3. As dez invariantes	8
3.1 1. Sem LLM em runtime	8
3.2 2. Ausência de dado não é insatisfação	8
3.3 3. Score, nota e categoria são derivados no servidor	8
3.4 4. Faixas de NPS: 0–6 detrator, 7–8 neutro, 9–10 promotor	9
3.5 5. Latência nunca é persistida	9
3.6 6. Timestamps timezone-aware	9
3.7 7. Modelo ausente é falha alta e explícita	9
3.8 8. A ordem das classes é 0 insatisfeito, 1 neutro, 2 satisfeito	10
3.9 9. As 39 chaves de feature batem exatamente com NOMES_FEATURES	10
3.10 10. O corpus de treino não pode entregar o rótulo	10
4. Arquitetura	11
4.1 A decisão que explica o desenho	11
4.2 As sete famílias que pontuam	12
4.3 A ironia continua carregada, e não pontua	12
4.4 Régua: o que pontuou uma conversa não é fixo no tempo	12
4.5 Repontuar é um trabalho, não uma requisição	12
4.6 Onde cada coisa mora	13
5. Treinamento dos modelos do Fraus	14
5.1 Como abrir no Colab	15
5.2 Notebook 01 — classificador de texto	15
5.2.1 Exigencia de GPU	15
5.2.2 Onde o checkpoint fica no Drive	15
5.2.3 Para onde copiar o artefato no repo local	15
5.2.4 Mapeamento de rótulo	16

5.3 Notebook 02 — fusor dos tres sinais	16
5.3.1 Rotulo do fusor: recommend_to_a_friend	16
5.3.2 Limitacao metodologica a declarar no relatorio	17
5.3.3 O vazamento do primeiro fusor — 13/08/2026	17
5.3.4 O fusor de 35 features — 24/08/2026	18
5.3.5 Consequencia no NPS: a classe neutra cai em detrator — HISTORICO (fusor de 16 features, ate 24/08/2026)	19
5.3.6 Retreino do fusor apos as features de incongruencia (03/09/2026)	19
5.3.7 A ironia sai do vetor (04/09/2026)	21
5.3.8 O fusor de 38 features — 04/09/2026	22
5.3.9 O contrato sobe para 39: incongruencia implicita (04/09/2026)	22
5.3.10 Conferencia do artefato	23
5.4 Contrato de features	23
5.5 Notebook 03 — classificador de emocao	24
5.5.1 Ordem canonica das classes	24
5.5.2 Corpus: go_emotions_ptbr	25
5.5.3 O desequilibrio, e por que NAO subamostrar	25
5.5.4 O teto do desprezo — a declarar no relatorio	25
5.5.5 Por que o desprezo NAO e treinado	26
5.6 Notebook 04 — classificador de ironia	26
5.6.1 O corpus NAO tem download aberto	26
5.6.2 Alternativas ao IDPT — levantamento de 14/08/2026	27
5.6.3 Onde existe corpus PT-BR de ironia (e como pedir)	27
5.6.4 O padrao: corpus sintetico	27
5.7 Sinal lexico — SentiLex-PT02	28
5.8 Sinal de estilo	28
5.9 Notebook 05 — calibração, e por que ele não retreina nada	29
5.10 Notebook 02 — a seção 10 é uma pendência, não uma etapa	29
5.10.1 As três saídas, medidas em 15/09/2026 — decidido: só log1p	30
5.10.2 Promovido em 15/09/2026	30
6. Fraus — Design System	31
6.1 1. A tese, e por que ela governa a composição	31
6.1.1 1.1 O que é notação e o que é decoração	32
6.2 2. Composição	33
6.2.1 2.1 O que sai	33
6.2.2 2.2 O que entra	34
6.3 3. Tokens	34
6.3.1 3.1 O fundo é papel, não vazio	35
6.3.2 3.2 As duas vozes	37

6.3.3	3.3 A cor da marca: o dourado do monograma	38
6.3.4	3.3.1 O raio acompanha os cortes do monograma	39
6.3.5	3.3.2 O rótulo dentro do botão é verificado	39
6.3.6	3.4 Categorias	39
6.4	4. Tipografia	40
6.4.1	4.1 A prosa metodológica vira aparato	40
6.5	5. Estados	41
6.6	6. Movimento	41
6.7	7. A materialidade	42
6.8	8. Os temas	43
6.8.1	8.1 A tese do tema chuva	43
6.8.2	8.2 O que o tema NÃO toca	44
6.8.3	8.3 O gate passou a ser ciente de tema	44
6.8.4	8.4 Onde a escolha mora	44
6.8.5	8.5 Onde a chuva pode gritar	45
6.8.6	8.6 O preço: o vidro da chuva é mais grosso	45
6.8.7	8.7 O padrão de fábrica: espaço profundo (01/09/2026)	50
6.9	9. O ciclo de verificação, e o que cada gate NÃO vê	51
7.	Conformidade regulatória — o Fraus e o EU AI Act	52
7.1	1. Por que este documento existe	52
7.2	2. Escopo do sistema	52
7.3	3. As três obrigações que já valem, e a que vem	52
7.3.1	3.1 Transparência — Art. 50(3), desde 02/08/2026	52
7.3.2	3.2 Proibição no local de trabalho — desde 02/02/2025	53
7.3.3	Regra: o Fraus não pontua atendentes.	53
7.3.4	3.3 Alto risco (Anexo III) — aplicação em 02/12/2027	53
7.4	4. Limitações que importam para quem lê um número do Fraus	53
7.5	5. Fontes	54
8.	Encolher os modelos — o que foi medido, e por que a int8 foi recusada	55
8.1	O problema	55
8.2	O que foi testado	55
8.3	O resultado da melhor variante	55
8.4	E é aqui que ela é recusada	56
8.5	A variante mista, medida em 11/09/2026	56
8.5.1	De quem é a culpa, então	56
8.5.2	E o que isso faz com a máquina de 1 GB	57
8.5.3	O que sobra de ganho real	57

8.6	O backend ONNX em produção, medido em 15/09/2026	58
8.6.1	Emoção com int8 só nos pesos – aceita pelo portão, não promovida por padrão	58
8.7	O que sobra como caminho	59
8.8	Reproduzir	59
8.9	Variante experimental: um encoder, três cabeças	59
9.	Limitações conhecidas	60
9.1	Limitações operacionais do deploy gratuito	60
9.2	O NPS é inferido, não perguntado	60
9.3	O sinal de tempo é treinado em dados sintéticos	60
9.4	A cabeça de ironia é pouco confiável	61
9.5	Desprezo não é uma classe treinada	61
9.6	Nenhuma feature agregada reverte uma probabilidade saturada	61
9.7	O que não dá para consertar sem corpus de atendimento real	61
10.	Referência de código	62
10.1	Indicadores	62
10.1.1	fraus.indicadores	62
10.2	Modelo canônico	65
10.2.1	fraus.modelos	65
10.3	O fusor e as 39 features	66
10.3.1	fraus.fusor	66
10.4	Os sinais	69
10.4.1	Texto – o único que pontua sozinho	69
10.4.2	fraus.sinais.texto	69
10.4.3	Emoção	69
10.4.4	fraus.sinais.emocao	69
10.4.5	Tempo	70
10.4.6	fraus.sinais.tempo	70
10.4.7	Léxico	71
10.4.8	fraus.sinais.lexico	71
10.5	Evidência e deriva	72
10.5.1	Evidência fraca – a cabeça tracejada	72
10.5.2	fraus.evidencia	72
10.5.3	Deriva de distribuição – a invariante 10 em runtime	72
10.5.4	fraus.deriva	72

1. Fraus

Ferramenta que lê atendimento por chatbot e **estima satisfação sem perguntar nada ao cliente**.

A tese é o caso que uma pesquisa de NPS declarada não captura: o cliente que escreve "ok, obrigado 😊" e sai insatisfeito. Ele não responde pesquisa — e é justamente ele que o sistema precisa enxergar.

⚠️ Todo número aqui é inferido

O NPS do Fraus **não é perguntado**: ele é derivado do texto do atendimento. Toda exibição carrega a etiqueta de estimativa, e apresentá-lo como NPS declarado seria falso. Ver [Limitações](#) e [Conformidade](#).

1.1 O que ler primeiro

Se você quer	Leia
entender a aposta do projeto	A tese
mexer no código	As invariantes — comece por aqui, sem exceção
saber como as peças se ligam	Arquitetura
saber como os modelos foram treinados	Treinamento
entender a interface	Design
saber o que o sistema não sabe	Limitações
entender como a aplicação real foi hospedada	Deploy gratuito

1.2 Sem LLM em runtime

Três BERTimbau fine-tunados (satisfação, emoção, ironia) e um fusor `LogisticRegression` sobre 39 features de sete famílias de sinal. Inferência local, em CPU, sem nenhuma chamada de rede no caminho de predição.

Isso é **requisito**, não otimização: o trabalho existe para demonstrar que dá para medir isso com modelos pequenos e interpretáveis — coeficiente de regressão logística responde *por que* um atendimento recebeu a nota, e floresta densa não.

1.3 Baixar

O PDF com o site inteiro está em [pdf/fraus.pdf](#), gerado no mesmo build que publica estas páginas — então ele nunca está mais velho que o site.

2. A tese

2.1 O problema

Quem opera um chatbot de atendimento mede satisfação de dois jeitos, e os dois falham no mesmo ponto.

Pesquisa declarada (NPS, CSAT). Depende de resposta, e a taxa de resposta é baixa e enviesada: responde quem ficou muito irritado ou muito satisfeito. O cliente do meio — o que resolveu o problema com esforço, ou desistiu no meio — simplesmente não aparece.

Taxa de contenção. Mede se o bot segurou o atendimento sem escalar. É um número de **volume**, não de qualidade: sobe igual quando o bot resolve e quando o cliente desiste de tentar.

Cruzados, os dois produzem o pior cenário possível: contenção alta com satisfação não medida. O painel fica verde e ninguém sabe se a operação está boa.

2.2 A aposta

O texto do atendimento **já contém** a evidência da satisfação. Ela não precisa ser perguntada — precisa ser lida.

O caso canônico do projeto é este:

| "que atendimento maravilhoso, só esperei 3 horas"

Uma pesquisa nunca vai receber essa frase. Um classificador de sentimento ingênuo lê "maravilhoso" e marca promotor. O sistema precisa das duas metades ao mesmo tempo: o que foi **dito** e o que foi **medido**.

2.3 O que sai disso, e que nada mais consegue calcular

Falso containment. O cruzamento de "não escalou" com "saiu detrator". É a tese virando número, e só quem infere satisfação do texto consegue calcular: quem depende de pesquisa respondida perde justamente o cliente contido, que é o que não responde.

2.4 O que a tese NÃO promete

- **Não é NPS declarado.** É estimativa, e a interface diz isso em todo lugar.
- **Não decide nada sozinha.** Nenhuma saída aciona ação sobre ninguém.
- **Não pontua atendentes** — ver [Conformidade](#).
- **Não é infalível, e diz onde falha.** O sinal de tempo é treinado em dados sintéticos; a cabeça de ironia é pouco confiável e por isso não pontua; fora da faixa de latência do treino o relógio domina a nota. Tudo isso está em [Limitações](#), com número.

3. As dez invariantes

Não são preferências de estilo. **Cada uma nasceu de um defeito real**, e violá-la é reintroduzir o defeito — por isso cada regra aqui vem com a cicatriz que a produziu. Documentação que explica cicatriz é a que não dá para copiar de tutorial.

O formato é sempre o mesmo: **a regra**, *por que ela existe*, **o que quebra se alguém a violar**, e onde ela mora no código.

3.1 1. Sem LLM em runtime

A regra. Inferência local em CPU. Nenhuma chamada de rede no caminho de predição.

Por que existe. É requisito do trabalho, não otimização de custo. A tese é que dá para medir satisfação com modelos pequenos, fine-tunados e interpretáveis — chamar uma API de LLM não deixaria o sistema mais rápido, e invalidaria o que ele se propõe a demonstrar.

O que quebra. O trabalho inteiro. Nenhum teste pega isso: o código funcionaria melhor e a defesa cairia.

Onde mora. Três BERTimbau em `fraus/sinais/` e um `LogisticRegression` em `fraus/fusor.py`.

3.2 2. Ausência de dado não é insatisfação

A regra. Conversa sem fala do cliente tem `score: None` e aparece como "sem sinal" — nunca `0`, em lugar nenhum: célula, gráfico, ordenação, export, agregado.

Por que existe. `None` e `0` significam coisas diferentes e não podem se confundir. Zero numa coluna de tempo de resposta se lê como "respondeu na hora", e a conversa que nunca teve atendente humano apareceria como **a mais ágil da operação**. Zero num NPS se lê como um número que alguém mediu.

O que quebra. Silenciosamente, tudo: o pior valor da escala aparece onde a verdade era "não medi". Em 10/09/2026 essa regra ainda estava furada dentro do próprio arquivo que mais a defende — `containment_rate` devolvia `0.0` no conjunto vazio, e o front tinha uma guarda para não acreditar nele.

Onde mora. Em toda parte. Procure `?? 0` e `|| 0` antes de commitar — e **também `return 0` no Python**, que é o mesmo defeito disfarçado de inicialização.

3.3 3. Score, nota e categoria são derivados no servidor

A regra. Nunca aceitos do corpo da requisição, nunca recalculados no cliente.

Por que existe. O `round` do Python é bancário (`round(6.5) == 6`) e o `Math.round` do JavaScript arredonda meio para cima (`Math.round(6.5) == 7`).

O que quebra. Já quebrou: a tabela exibia **nota 7 ao lado da etiqueta "Detrator"**, nas fronteiras 6/7 e 8/9. Duas fontes da verdade para o mesmo número, discordando na tela do usuário.

Onde mora. `fraus/indicadores.py::nota_0_10`. O front consome; não recalcula.

3.4 4. Faixas de NPS: 0–6 detrator, 7–8 neutro, 9–10 promotor

A regra. Padrão de fábrica em `FAIXAS_NPS`, configurável por `PUT /configuracoes` — e só se as faixas cobrirem 0..10 de forma contígua. A faixa vigente é passada **por parâmetro**, nunca lida de estado global.

Por que existe. Estado global mutável muda sob os pés de quem já calculou. E faixa com buraco ou sobreposição não é preferência de gosto: é nota sem categoria, ou com duas.

O que quebra. Já quebrou: `categoriaDaNota` no front tinha `<= 6` e `<= 8` digitados no corpo, e isso rodava no caminho feliz. Um operador que mexesse em Configurações via a **mesma tela dar dois vereditos** para o mesmo atendimento — a etiqueta da linha com as faixas novas, a cor da barra com as antigas.

Onde mora. `fraus/indicadores.py::FAIXAS_NPS`, `categoria_nps`, `validar_faixas_nps`.

3.5 5. Latência nunca é persistida

A regra. Sempre derivada dos timestamps, na leitura.

Por que existe. Latência gravada é latência que envelhece: qualquer correção na definição de "espera" deixaria o banco cheio de números calculados pela regra antiga, indistinguíveis dos novos.

O que quebra. Duas definições de latência convivendo — uma para a lista, outra para o detalhe — e as duas telas discordando sobre o mesmo atendimento.

Onde mora. `fraus/sinais/tempo.py::latencias_da_conversa`.

3.6 6. Timestamps timezone-aware

A regra. `datetime` naive é erro de validação, não conversão silenciosa.

Por que existe. Data sem fuso é data que muda de significado conforme quem lê.

O que quebra. A armadilha já paga: **a data da Totalk é MM/DD/YYYY**, não DD/MM. Lida como brasileira, espalha as mensagens por cinco meses e a latência sai absurda **em silêncio**.

Onde mora. `fraus/modelos.py`.

3.7 7. Modelo ausente é falha alta e explícita

A regra. Sem fallback, sem motor dublê silencioso. A API não sobe sem os três modelos treinados.

Por que existe. Servir predição sem modelo carregado é **pior que estar fora do ar**: a interface não distingue número real de número sintético.

O que quebra. Já quebrou, e custou um dia: o `scripts/api_demo.py` estava no ar por causa do diretório de onde foi lançado, e a dashboard escreveu "API no ar" durante um dia inteiro enquanto **todo número da tela era sintético** — os pesos por feature saíam numa progressão 0,2 / 0,25 / 0,3 e passavam por coeficiente de regressão logística.

Onde mora. `Fusor.carregar` (valida `n_features_in_`), `GET /saude ("motor": "real" | "duble")`, e `GET /saude/deriva`, que recusa diagnosticar com dublê pela mesma lógica.

3.8 8. A ordem das classes é 0 insatisfeito, 1 neutro, 2 satisfeito

A regra. No notebook, no sinal de texto, no fusor e nos indicadores.

Por que existe. É a única convenção que amarra quatro lugares que não se importam por código.

O que quebra. Nada estoura. O sistema pontua ao contrário, em silêncio, com a mesma confiança de sempre. É a invariante mais barata de violar e a mais cara de descobrir.

Onde mora. `fraus/sinais/texto.py` (`INSATISFEITO` , `NEUTRO` , `SATISFEITO`).

3.9 9. As 39 chaves de feature batem exatamente com `NOMES_FEATURES`

A regra. As sete famílias do vetor produzem exatamente 39 chaves. `vetorizar` levanta `KeyError` na falta — nunca zero silencioso.

Por que existe. Zero numa feature ausente não é neutro: o modelo aprendeu o que zero significa naquela dimensão, e a conversa entraria como se tivesse sido medida.

O que quebra. Já quebrou de outro jeito: o `CLAUDE.md` chegou a declarar "38" na invariante e "(35)" no mapa — dois números de duas revisões, nenhum o vigente. A documentação que governa toda sessão de agente envelhecendo em silêncio. Hoje há teste amarrando o número do `CLAUDE.md` a `len(NOMES_FEATURES)`.

Onde mora. `fraus/fusor.py::NOMES_FEATURES` — e o comentário dele conta a história inteira: $40 - 2$ (a ironia saiu) + 1 (`incongruencia_situacao_negativa` entrou) = 39.

3.10 10. O corpus de treino não pode entregar o rótulo

A regra. Distribuição por rótulo se sobrepõe; feature constante no treino nasce com peso zero. **Acurácia alta demais é sintoma, não vitória.**

Por que existe. Faixa de latência disjunta por classe fez o primeiro fusor marcar **99,3% lendo só o relógio**, com o BERTimbau apagado.

O que quebra. Um número excelente que não mede nada. E ela não para no treino: em 08/09/2026 a mesma invariante apareceu **em runtime** — numa conversa com 3 h de espera, a latência contribuiu $-85,4$ contra $+4,6$ do texto inteiro, e o score saiu em $4,4e-24$. O corpus nunca passou de minutos, então a guarda de vazamento, que olha o corpus, não tinha o que dizer.

Onde mora. `docs/treinamento.md`, e agora também `GET /saude/deriva`, que transforma a regra de disciplina de notebook em alarme de runtime.

4. Arquitetura

```

flowchart TD
    A["CSV / Discord / WhatsApp / webhook"] -->|"adapter de ingestão<br/>fraus/ingest/"| B["Conversa<br/><small>modelo canônico único – fraus/modelos.py</small>"]

    B --> texto["texto<br/><small>BERTimbau por mensagem</small>"]
    B --> emoji["emoji<br/><small>lexicon + posição</small>"]
    B --> tempo["tempo<br/><small>latência, escalação</small>"]
    B --> emocao["emoção<br/><small>7 classes + desprezo</small>"]
    B --> lexico["léxico<br/><small>SentiLex-PT02 + negação</small>"]
    B --> estilo["estilo<br/><small>caixa alta, palavra</small>"]
    B --> incong["incongruência<br/><small>emoji × texto</small>"]
    B --> ironia["ironia<br/><small>cabeça binária – só tela</small>"]

    texto --> V["39 features"]
    emoji --> V
    tempo --> V
    emocao --> V
    lexico --> V
    estilo --> V
    incong --> V

    V --> F["Fusor<br/><small>LogisticRegression + StandardScaler</small>"]
    F -->|"score 0–100"| N["nota 0–10 → categoria NPS → indicadores agregados"]
    N --> API["FastAPI"]
    API --> D["dashboard Next.js"]
    ironia -->|"por mensagem, direto pra tela"| D

    style ironia stroke-dasharray: 4 4
  
```

* a cabeça de ironia é lida por mensagem e mostrada na tela, mas não entra nas 39 features do Fusor — ver a seção abaixo.

4.1 A decisão que explica o desenho

Um modelo canônico, muitos adaptadores. CSV, Totalk, transcrição de `.docx` / `.pdf` e webhook desembocam todos em `Conversa` — e o resto do sistema não sabe de onde o dado veio. Adicionar uma fonte é escrever um adaptador, não tocar em sinal, fusor ou indicador.

4.2 As sete famílias que pontuam

família	o que mede	por que existe
texto	P(insatisfeito/neutro/satisfeito) por mensagem	o sinal principal — o único fine-tunado para a tarefa
emoji	polaridade e posição relativa	emoji no fim da conversa não vale o mesmo que no começo
tempo	latências, escalação, abandono	espera longa degrada satisfação (literatura de live chat)
emoção	7 de Ekman + desprezo derivado	raiva e frustração distinguem <i>tipos</i> de insatisfação
léxico	SentiLex-PT02 com escopo de negação	âncora determinística, auditável, imune a retreino
estilo	caixa alta, alongamento, palavrão, censura	intensidade que o classificador de sentença perde
incongruência	elogio convivendo com situação negativa	é o que alcança a frase canônica do projeto

4.3 A ironia continua carregada, e não pontua

A oitava cabeça existe, é obrigatória para a API subir e aparece na tela **por mensagem** — mas **saiu do vetor** em 04/09/2026.

Medida no próprio corpus de treino do fusor (B2W-Reviews01), ela marca **74% das resenhas satisfeitas** como irônicas contra **9% das insatisfeitas**: naquele corpus ela é um detector de sentimento **positivo**, não de ironia. O fusor aprendeu peso **+0,77** para `ironia_prob_media`, empurrando ironia para *satisfeito* — o inverso do que o nome promete.

Feature que mede outra coisa que não o nome dela é pior que feature ausente. Ela some do vetor sem que ninguém perceba a inversão, porque a acurácia global continua ótima.

Ver [Treinamento](#) para a medição completa.

4.4 Régua: o que pontuou uma conversa não é fixo no tempo

Cada conversa é gravada com a **régua** que a pontuou: a assinatura do motor servindo (`Motor.regua`, `None` no motor dublê) mais a versão da curadoria (`lexico_versao`). Trocar de checkpoint, trocar de backend (torch/onnx) ou o analista ensinar um termo novo ao léxico muda a régua — e o banco passa a ter conversas antigas pontuadas por uma régua que não é mais a vigente.

O sistema não recalcula sozinho: ele **avisa**. `GET /indicadores` expõe `pontuadas_com_regua_antiga`, contando contra a régua vigente lida a cada requisição (nunca guardada em memória, pelo mesmo motivo de `faixas_vigentes` e `curadoria_vigente` — sem cópia de estado envelhecendo). A tela mostra o aviso com o motivo (checkpoint novo, léxico curado, ou os dois), nunca um número solto sem explicação.

4.5 Repontuar é um trabalho, não uma requisição

Zerar o aviso de régua misturada significa rodar os três BERTimbau de novo em cada conversa do banco — em milhares de atendimentos isso estoura qualquer timeout HTTP. `POST /conversas/repontuar` (`fraus/api/repontuacao.py`) só dispara uma thread em segundo plano; `GET /conversas/repontuar` lê o progresso (`total`, `feitas`, `estado`, `erro`) que a tela transforma em barra. Não é

fila — é **um trabalho por vez**: uma segunda chamada enquanto a primeira roda recebe 409, porque duas repontuações em paralelo gravariam o mesmo banco com duas curadorias diferentes. O estado mora na memória do processo (reiniciar a API no meio perde o progresso, não o que já foi gravado — cada conversa salva já carrega sua própria `lexico_versao` e régua).

```
sequenceDiagram
    actor T as Tela
    participant API as FastAPI
    participant R as Repontuacao (thread)
    participant M as Motor (3x BERTimbau)
    participant DB as banco

    T->>API: POST /conversas/repontuar
    API->>R: iniciar(ctx)
    Note over R: le curadoria, faixas e régua<br/>UMA vez – janela fechada
    R->>API: estado=rodando, total=N
    API->>T: 202 (ou 409 se já rodando)
    R->>M: dispara thread em segundo plano

    loop cada conversa do lote
        M->>M: pontuar_conversa
        M->>DB: salvar(score, lexico_versao, régua)
        R->>R: feitas += 1
    end

    loop enquanto roda
        T->>API: GET /conversas/repontuar
        API->>T: {total, feitas, estado}
        Note over T: barra de progresso
    end

    R->>API: estado=concluido (ou falhou)
    T->>API: GET /conversas/repontuar
    API->>T: estado final
```

4.6 Onde cada coisa mora

caminho	responsabilidade
<code>fraus/modelos.py</code>	Conversa / Mensagem — o modelo canônico
<code>fraus/ingest/</code>	adaptadores de entrada, um por formato
<code>fraus/sinais/</code>	as oito famílias de sinal
<code>fraus/sinais/curadoria.py</code>	o que o analista ensinou ao léxico — vence o SentiLex
<code>fraus/fusor.py</code>	<code>NOMES_FEATURES</code> (39) e o <code>Fusor</code>
<code>fraus/indicadores.py</code>	NPS, CSAT, contenção, falso containment, série
<code>fraus/evidencia.py</code>	evidência fraca — a cabeça tracejada
<code>fraus/deriva.py</code>	deriva de distribuição — a invariante 10 em runtime
<code>fraus/cortesia.py</code>	"ok, obrigado" sozinho é sem sinal, não promotor
<code>fraus/api/</code>	FastAPI: <code>criar_app</code> (fábrica) e as rotas por domínio
<code>fraus/api/contexto.py</code>	faixas, curadoria e régua vigentes, lidas por requisição
<code>fraus/api/repontuacao.py</code>	repontuar em segundo plano, com progresso e 409 se já rodando
<code>fraus/api/vazao.py</code>	teto de <code>/auth/*</code> por IP e de <code>/ingestao</code> por fonte
<code>dashboard/</code>	Next.js — ver Design

5. Treinamento dos modelos do Fraus

#	Notebook	O que produz	GPU?	Estado
1	<code>notebooks/01_treino_bertimbau.ipynb</code>	<code>modelos/bertimbau-satisfacao/</code> — classificador de texto (3 classes), fine-tune do <code>neuralmind/bert-base-portuguese-cased</code> sobre o B2W-Reviews01	sim	pronto
2	<code>notebooks/02_treino_fusor.ipynb</code>	<code>modelos/fusor.joblib</code> e <code>modelos/importancias.json</code> — a regressao logistica que funde os sinais	nao	pronto
3	<code>notebooks/03_treino_emocao.ipynb</code>	<code>modelos/bertimbau-emocao/</code> e <code>modelos/metricas_emocao.json</code> — classificador das sete classes de emocao	sim	pronto
4	<code>notebooks/04_treino_ironia.ipynb</code>	<code>modelos/bertimbau-ironia/</code> e <code>modelos/metricas_ironia.json</code> — classificador binario de ironia	sim	pronto, mas o corpus precisa ser solicitado

O notebook 02 **depende do artefato do 01**: ele carrega o BERTimbau treinado para extrair o sinal de texto de cada conversa. A primeira celula do 02 falha com erro explicito se `modelos/bertimbau-satisfacao` nao estiver no Drive.

Os notebooks 03 e 04 sao **independentes entre si e do 01** — treinam cabecas separadas, em corpora proprios. O 02 e que passa a depender dos tres, porque e ele que monta o vetor de features completo. Ordem de execucao: 01, 03 e 04 em qualquer ordem, e o 02 por ultimo.

O sinal lexico (`fraus/sinais/lexico.py`) **nao tem notebook**: o SentiLex-PT nao e treinado, e recurso pronto, versionado em `fraus/dados/sentilex_pt02.csv`. Ver [Sinal lexico](#) no fim.

Os dois artefatos sao obrigatorios para o app real subir. `fraus/api/main.py` carrega o classificador e o fusor no boot e propaga o erro sem fallback — servir predicao sem modelo carregado e pior do que estar fora do ar. Com so um dos dois, o unico servidor que sobe e `scripts/api_demo.py`, que usa motor duble e nao serve para producao.

O treino roda inteiramente no Google Colab, fora deste repositorio — nao ha teste automatizado local para essas etapas.

5.1 Como abrir no Colab

1. Va em colab.research.google.com.
2. Arquivo > Fazer upload de notebook e selecione o notebook (ou abra direto do GitHub se o repositório estiver acessível a partir da sua conta).

5.2 Notebook 01 — classificador de texto

5.2.1 Exigencia de GPU

O notebook só funciona com acelerador de GPU habilitado. Antes de rodar qualquer célula:

```
Ambiente de execucao > Alterar tipo de ambiente de execucao > GPU
```

A primeira célula do notebook faz `assert torch.cuda.is_available()` e falha com uma mensagem explícita se o ambiente de execução estiver sem GPU.

5.2.2 Onde o checkpoint fica no Drive

O notebook monta o Google Drive (`drive.mount('/content/drive')`) e usa dois diretórios dentro dele, para sobreviver a uma queda de sessão do Colab — o mesmo padrão já usado no notebook de RVC do projeto Neuro-ai:

- `/content/drive/MyDrive/fraus/checkpoints` — checkpoints intermediários salvos pelo `Trainer` a cada época (`save_strategy="epoch", save_total_limit=2`). Se a sessão cair, o próximo `Trainer.train()` pode retomar a partir do último checkpoint salvo aqui.
- `/content/drive/MyDrive/fraus/modelos/bertimbau-satisfacao` — destino final do modelo treinado, do tokenizador e do `metricas.json`, escritos na última célula do notebook.

5.2.3 Para onde copiar o artefato no repo local

Depois que o notebook terminar (célula de avaliação e export), baixe a pasta `fraus/modelos/bertimbau-satisfacao` do Google Drive e copie o conteúdo para:

```
modelos/bertimbau-satisfacao/
```

na raiz deste repositório. Os arquivos esperados são:

- `config.json`
- `model.safetensors`
- `tokenizer.json`
- `tokenizer_config.json`
- `special_tokens_map.json`
- `metricas.json` — `{"acuracia": float, "f1_macro": float, "classes": ["insatisfeito", "neutro", "satisfeito"]}`

`fraus/sinais/texto.py` carrega esse diretório para servir o classificador — e o notebook 02 também, para extrair o sinal de texto das conversas de treino do fusor.

Importante: `modelos/` está no `.gitignore` deste repositório. O artefato do modelo treinado (pesos, tokenizador, métricas) **nao vai para o git** — ele fica local, versionado apenas via o checkpoint no Drive.

5.2.4 Mapeamento de rotulo

O rotulo de satisfacao vem do campo `overall_rating` (nota de 1 a 5) do B2W-Reviews01:

<code>overall_rating</code>	rotulo	indice
1-2	insatisfeito	0
3	neutro	1
4-5	satisfeito	2

O corpus e balanceado por subamostragem da classe majoritaria antes do split de treino/teste, porque as notas 4-5 dominam o corpus e enviesariam o classificador.

5.3 Notebook 02 — fusor dos tres sinais

Roda DEPOIS do 01, sem GPU. O que ele faz, em sequencia:

1. monta o Drive e confere `modelos/bertimbau-satisfacao` (falha alto apontando o notebook 01 se faltar);
2. clona este repositorio e instala o pacote `fraus` — a extracao de features usa o MESMO codigo da API (`fraus.fusor.montar_features`), nunca uma reimplementacao;
3. carrega o B2W-Reviews01 e rotula por `recommend_to_a_friend` (ver abaixo);
4. costura as frases em conversas sinteticas com `fraus.ingest.simulador.gerar_lote`, deterministico por semente, com latencia log-normal e emoji calibrados por rotulo;
5. extrai as features de cada conversa com o BERTimbau do notebook 01 carregado — hoje sao **39** (ver [Contrato de features](#));
6. treina o Fusor (`treinar(exemplos, rotulos)`);
7. avalia num conjunto de teste separado — conversas geradas com outra semente e a partir de frases disjuntas — imprimindo acuracia e F1-macro;
8. exporta `fusor.joblib` (via `Fusor.salvar`) e `importancias.json` (o retorno de `Fusor.importancias()`), que vira o grafico "qual sinal pesou mais" da apresentacao).

Os dois arquivos saem em `/content/drive/MyDrive/fraus/modelos/`. Baixe para `modelos/` na raiz do repositorio local, ao lado de `bertimbau-satisfacao/`.

5.3.1 Rotulo do fusor: `recommend_to_a_friend`

Esta e a etapa em que o campo `recommend_to_a_friend` do B2W-Reviews01 entra. A spec (secao 5.1) o declara como a **âncora do NPS inferido** — "sem ele, o NPS da dashboard seria metrica inventada" — e o alvo do fusor e justamente a categoria de NPS que a dashboard exhibe. O notebook 01 usou `overall_rating` porque treinava sentimento de texto; o 02 usa a recomendacao porque treina NPS.

campo	valor	classe	indice
<code>recommend_to_a_friend</code>	Yes	satisfeito	2
<code>recommend_to_a_friend</code>	No	insatisfeito	0
<code>overall_rating</code>	3	neutro	1

Por que a linha do neutro existe. `recommend_to_a_friend` e binario, e as classes do projeto sao tres. Um fusor de duas classes zeraria o termo do neutro no score ($100 * (P(\text{satisfeito}) + 0.5 * P(\text{neutro}))$) e empurraria todo atendimento morno para uma das pontas, inflando o NPS nos dois sentidos. A alternativa seria treinar em duas classes e recortar o neutro por uma faixa de score — mas isso trocaria um rotulo observado por um limiar arbitrado a mao depois do treino. Aqui o neutro e `overall_rating == 3`, um

rotulo do corpus, e a nota 3 tem precedencia sobre a recomendacao para que a mesma resenha nao caia em duas classes. As duas pontas — as unicas que contam no NPS, ja que neutros nao entram na formula — seguem ancoradas em `recommend_to_a_friend`.

As tres classes sao subamostradas ao tamanho da menor (a nota 3 e minoria no corpus), o mesmo balanceamento do notebook 01.

5.3.2 Limitacao metodologica a declarar no relatorio

O fusor e treinado sobre conversas SINTETICAS: nenhum corpus publico de resenha PT-BR tem timestamps de dialogo, entao a latencia, a escalacao e o abandono saem de distribuicoes calibradas pela literatura de live chat, nao de atendimento observado. O texto e real (B2W), o tempo nao e.

As metricas do notebook 02 continuam OTIMISTAS mesmo depois da correcao descrita abaixo: elas medem que o fusor aprendeu a combinar os sinais no dominio sintetico, nao que ele acertaria essa taxa em atendimento real.

5.3.3 O vazamento do primeiro fusor — 13/08/2026

O primeiro fusor treinado marcou **99,3% de acuracia e F1-macro 0,993**, e era inutil. Vale registrar porque o modo de falha e silencioso e pode voltar.

O sintoma. Carregado no repositorio local e alimentado com conversas escritas a mao, o fusor devolvia praticamente a mesma nota para tudo:

	latencia 5s	30s	200s
texto otimo (BERTimbau: 0,96 satisfeito)	52,48	51,32	50,02
texto pessimo (BERTimbau: 0,98 insatisfeito)	49,94	49,92	49,84

Dois textos que o classificador de texto separa com mais de 95% de confianca saiam com **2,5 pontos** de diferenca, ambos colados em 50 e ambos classificados como neutro.

A causa. O `PERFIL_POR_ROTULO` do simulador amostrava latencia de faixas **disjuntas** — insatisfeito 60-400s, neutro 15-60s, satisfeito 3-15s. Sem sobreposicao nenhuma, a latencia *era* o rotulo: dava para acertar a classe olhando so o relógio, sem ler uma letra do texto. A regressao logistica fez exatamente isso. Os quatro maiores pesos aprendidos eram todos de tempo ($\text{latencia_p90_s} \cdot 2,06 + \text{latencia_mediana_s} \cdot 2,03 + \text{latencia_primeira_resposta_s} \cdot 1,91 + \text{duracao_total_s} \cdot 1,52$), contra 1,39 e 1,12 das features de texto. Fora da variedade que ele decorou, o modelo respondia "neutro" para tudo.

Os 99,3% nao mediam fusao. Mediam que o simulador vazava o gabarito.

Um segundo achado, do mesmo diagnostico. As features `emoji_score_medio`, `emoji_frac_positivos` e `emoji_frac_negativos` tinham coeficiente **0,0000**: o simulador nunca emitia emoji, entao elas eram constantes no treino e um dos tres sinais da arquitetura nao contribuia nada.

A correcao (`fraus/ingest/simulador.py`, coberta por `tests/test_simulador.py`):

- a latencia passou a ser **log-normal** com caudas que se cruzam. As medianas continuam ordenadas, como manda a literatura de live chat, mas ~11% das conversas satisfeitas sao mais lentas que a mediana das insatisfeitas, e vice-versa. Existe atendimento rapido que termina mal;
- as falas do cliente passaram a levar **emoji do lexicon**, com cruzamento deliberado — cliente irritado manda 🙄 (+0,418), cliente satisfeito manda 😊 (-0,368) — para o emoji nao virar o proximo gabarito.

Como saber se voltou. Acuracia alta e sintoma que pede investigacao, nao veredito. Depois da correcao o notebook 02 marcou **0,96**, e desta vez o numero se sustentou: os pesos aprendidos mudaram de lugar.

feature	fusor vazado	fusor corrigido
emoji_score_medio	0,0000 (morto)	1,6662 (1º)
texto_prob_satisfeito_media	1,3858	1,6026 (2º)
texto_prob_insatisfeito_media	1,1212	1,4722 (3º)
latencia_p90_s	2,0575 (1º)	0,0674
latencia_mediana_s	2,0329	0,1621
latencia_primeira_resposta_s	1,9068	0,0138

O mesmo teste de sanidade que expôs o vazamento — mesmo texto, so mudando o relógio — passou de 2,5 pontos de separação para **99,6**: texto ótimo pontua 99,98 e texto péssimo 0,35, com a latência mexendo frações de ponto. O sinal de tempo virou o modificador fraco que ele deve ser.

O critério, então, não é o número sozinho. Diante de acurácia alta, olhe os PESOS: se as features que lideram forem as que carregam o conteúdo (texto, emoji), o modelo aprendeu; se forem as circunstanciais (tempo, contagem de turnos), procure o vazamento.

5.3.4 O fusor de 35 features — 24/08/2026

O contrato subiu de 16 para 35 features em 21/08 (emoção, léxico, ironia e estilo entraram no vetor) e o **artefato ficou três dias para trás**. Nesse intervalo a API real não subia: o `StandardScaler` de 16 rejeitava o vetor de 35 na carga. Falha alta e explícita, que é o comportamento que as invariantes 7 e 9 pedem — mas ainda assim três dias de API parada por artefato desatualizado.

O notebook 02 rodou de novo e o artefato daquele momento passou a ter `n_features_in_ = 35`, na ordem exata de `NOMES_FEATURES`. (Histórico: o artefato vigente hoje tem 38 — ver as seções de 03/09 e 04/09 abaixo.)

A acurácia caiu de 0,96 para 0,93, e a queda é o resultado saudável. O fusor de 16 features media um problema mais fácil. Dezenove features novas entraram, três delas vindas de uma cabeça de ironia com vazamento de corpus conhecido e não consertado — um número MENOR e o esperado. Pela regra desta seção, o que decide não é o número e sim onde os pesos foram parar:

feature	fusor de 16	fusor de 35
texto_prob_satisfeito_media	1,6026 (2º)	1,6572 (1º)
texto_prob_insatisfeito_media	1,4722 (3º)	1,3867 (2º)
emoji_score_medio	1,6662 (1º)	1,3354 (3º)
emoji_frac_positivos	0,8430	0,8098
emocao_surpresa_media	—	0,6655
emocao_desprezo_derivado	—	0,4757
latencia_mediana_s	0,1621	0,1200
latencia_p90_s	0,0674	0,1419
duracao_total_s	0,4964	0,3397

O topo continua sendo **texto e emoji**, e as features de tempo continuam no rodape, onde devem estar. As de emocao entram no meio da tabela com peso real sem deslocar o conteudo — `desprezo_derivado`, que e a diade calculada e nao uma classe do modelo, puxa 0,48. Nao ha sinal de vazamento novo.

A divida que este fusor assume. Ele foi treinado com a cabeca de ironia vazando (ver a pendencia 1 do README: 6 em 10 frases sinceras saem marcadas como ironicas). `ironia_prob_media` e `ironia_prob_max` pesam 0,30 e 0,31 — nao e desprezivel.

Retreinar a ironia obriga a retreinar o fusor de novo, e essa ordem esta registrada aqui para nao se perder.

Teste de sanidade ponta a ponta, com os tres BERTimbau carregados, sobre conversas do simulador: rotulo insatisfeito pontua ~0, neutro ~73-75, satisfeito ~99, e as tres categorias de NPS saem certas em 9 de 9. **Isso nao e evidencia de qualidade** — sao conversas do proprio gerador sintetico, o dominio em que o fusor foi treinado. O numero honesto continua sendo 0,93 no conjunto de teste separado, e a ressalva do topo desta secao continua valendo: as metricas do notebook 02 medem fusao no dominio sintetico, nao acerto em atendimento real.

5.3.5 Consequencia no NPS: a classe neutra cai em detrator — HISTORICO (fusor de 16 features, ate 24/08/2026)

Esta secao mede o fusor de 16 features, NAO o vigente. Ela ficou aqui sem data enquanto o artefato foi trocado, e passou a ler como se descrevesse o fusor atual — que e o defeito mais caro deste projeto. Fica registrada porque o raciocinio sobre o score continua valendo, e porque a medicao dela e o contraponto do que o fusor de 35 features entrega.

A consequencia DEIXOU de valer com o fusor de 35 features. Base: o teste de sanidade ponta a ponta da secao anterior, com os tres BERTimbau carregados — rotulo neutro pontua ~73-75, cai na faixa 7-8 e as tres categorias de NPS saem certas em 9 de 9. Ou seja, o neutro nao cai mais em detrator. Ressalva do mesmo tamanho: aquele teste corre sobre conversas do proprio simulador, o dominio em que o fusor foi treinado, entao ele desmente a medicao abaixo no dominio sintetico e nao promete nada sobre atendimento real.

Medido em 90 conversas do simulador, 30 por classe, o fusor corrigido separa bem — medianas **0,11 / 50,99 / 99,03** por rotulo verdadeiro. Mas a categoria de NPS derivada sai **67% detrator · 29% promotor · 4% neutro**, com **NPS -38** num lote equilibrado por construcao.

Nao e falha do treino. O score e $100 * (P(\text{satisfeito}) + 0.5 * P(\text{neutro}))$, entao conversa classificada com certeza como neutra pontua 50, vira nota 5 e cai na faixa 0-6. A faixa neutra do NPS (7-8) so seria alcançada com $P(\text{satisfeito})$ entre 0,4 e 0,8 — um empate entre classes, nao uma neutralidade confiante. As tres classes do modelo e as tres categorias do NPS foram decididas em lugares separados e nunca se encontraram.

Decisao de projeto: manter e declarar. A alternativa avaliada era subir o peso do neutro de 0,5 para 0,75 (o centro da faixa passiva do NPS), o que faria neutro puro pontuar 75, virar nota 8 e cair em neutro — alinhando as tres classes as tres categorias. Ficou registrada aqui e na docstring de `Fusor.pontuar` para nao ser "corrigida" por engano.

5.3.6 Retreino do fusor apos as features de incongruencia (03/09/2026)

O contrato subiu de 35 para 40 features: a familia `incongruencia_*` (`incongruencia_polaridade`, `incongruencia_emoji_texto`, `incongruencia_marcador_contraste`, `incongruencia_hiperbole`, `incongruencia_aspas_ironicas`) entrou em `NOMES_FEATURES`, anexada ao FIM da lista para preservar a ordem das 35 antigas. O fusor salvo em `modelos/` foi treinado com 35 e `vetorizar` agora produz 40 — ele NAO serve mais. `Fusor.carregar` (`joblib.load` puro, sem validacao de forma) carrega esse artefato sem erro nenhum -- a API real SOBE com ele. O erro de dimensao no `StandardScaler` so aparece na primeira pontuacao de verdade (`/ingestao` ou `/conversas/importar`), como HTTP 500, nao na carga do servidor. Isso e risco de demonstracao ao vivo, nao uma trava de subida -- ver a tabela do Fusor no README para o detalhe medido. A intencao da invariante 7 (falha alta e explicita) so se cumpre na metade "explicita"; a metade "na carga" fica para o retreino resolver o descompasso, nao para uma validacao que este codigo ainda nao tem.

O corpus sintetico tambem mudou, e isso importa mais do que a contagem de features. `FRASES_POR_ROTULO`, em `fraus/ingest/simulador.py`, ganhou frases novas em duas levas para curar dois vazamentos de rotulo — o mesmo tipo de falha silenciosa que ja custou o primeiro fusor (ver acima), so que desta vez em features que ainda nao estavam no vetor:

- **Intensificadores.** Antes, `incongruencia_hiperbole` era nao-zero na maior parte das conversas do rotulo satisfeito e ZERO nos rotulos insatisfeito e neutro — um previsor unilateral quase perfeito de "satisfeito". A literatura inclui hiperbole na deteccao de ironia justamente porque elogio hiperbolico e a forma classica da ironia, entao o fusor teria aprendido a feature ao contrario do que ela significa. Depois da correcao, medido com `gerar_lote(FRASES_POR_ROTULO, 180, semente=7)` — o mesmo lote e semente da guarda `test_nenhuma_feature_e_previsor_unilateral` em `tests/test_simulador.py`: `insatisfeito 34/60 | neutro 0/60 | satisfeito 39/60`.
- **Negacao.** `lexico_frac_negados` era nao-zero so no rotulo insatisfeito e ZERO nos outros dois — previsor unilateral de "insatisfeito", pre-existente desde antes da familia `incongruencia_*` e nunca notado por falta de guarda na familia `lexico_*`. Depois, medido com o mesmo lote e semente acima: `insatisfeito 15/60 | neutro 17/60 | satisfeito 21/60`.

Consequencia pratica: o notebook 02 precisa REGERAR o corpus sintetico, nao reusar um lote cacheado de uma execucao anterior. O notebook 02 ja gera o lote do zero a cada execucao — `gerar_lote` roda na propria celula, com semente fixa, e nao le nenhum artefato salvo de rodada passada — entao rodar o notebook de novo, sem alterar nada, ja traz o corpus corrigido. O risco e so para quem tiver guardado um `.csv` / `.pkl` de conversas de uma sessao antiga do Colab e tentar reaproveitar: um lote assim traz os dois vazamentos de volta.

O notebook ja importa o contrato do pacote, nao repete a lista. A celula marcada com o comentario

```
# 5. Os SETE sinais de cada conversa, pelo MESMO
codigo que a API usa (indice 8 do .ipynb, contando a partir de 0) faz from fraus.fusor import NOMES_FEATURES, montar_features e
chama montar_features com os tres classificadores — as cinco features novas entram sozinhas, sem editar o notebook. Isso ja era
assim antes desta rodada (o vazamento do ramo obsoleto registrado na celula 2 foi outra causa, a de importar de um ramo
desatualizado do repositorio, nao de reimplementar a lista) — nao ha lista duplicada para divergir aqui.
```

Passos, no Colab:

1. Conferir que a celula do clone do repositorio (RAMO) aponta para `main` com a Task 5 mergeada, e rodar o notebook do inicio — sem tentar reaproveitar nenhum lote de conversas salvo de uma execucao anterior.
2. Antes de subir ao Colab (ou depois, localmente), rodar as guardas de vazamento: `uv run pytest tests/test_simulador.py -k unilateral`. Elas cobrem as familias `lexico_*`, `emoji_*`, `estilo_*` e `incongruencia_*` contra previsor unilateral e precisam estar verdes antes do treino valer a pena.
3. Conferir a acuracia contra a do fusor de 35 features (0,93). **Acuracia alta demais e sintoma, nao vitoria** — se saltar muito acima do valor anterior, suspeite de vazamento antes de comemorar, releia a secao "O vazamento do primeiro fusor" acima e confira os PESOS, nao so o numero: se as features que lideram forem as de conteudo (texto, emoji), o modelo aprendeu; se forem as circunstanciais, procure o vazamento.
4. Conferir os coeficientes das cinco features novas em `Fusor.importancias()` / `Fusor.eixo_global()`.

CORRECAO DE 04/09/2026, e a licao vale mais que o numero: este passo dizia para esperar peso proximo de zero em `incongruencia_marcador_contraste` e `incongruencia_aspas_ironicas`, porque as duas sao constantes em zero no corpus do SIMULADOR. A previsao estava errada, e o erro foi confundir dois corpora diferentes. O simulador fornece a ESTRUTURA temporal (latencia, turnos, escalacao) a partir de seis frases por rotulo; o TEXTO do treino e o B2W-Reviews01, resenha real de e-commerce, onde marcador de contraste e aspas ironicas ocorrem normalmente. Medir uma feature de texto no gerador de estrutura e olhar para o corpus errado.

Os pesos reais do primeiro retreino de 40 features, no eixo satisfeito-menos-insatisfeito:

feature	peso
<code>incongruencia_emoji_texto</code>	-0,95
<code>incongruencia_hiperbole</code>	-0,65
<code>incongruencia_marcador_contraste</code>	-0,38
<code>incongruencia_polaridade</code>	-0,37
<code>incongruencia_aspas_ironicas</code>	-0,15

As cinco saíram NEGATIVAS, que é a direção que a literatura prevê: mais incongruência empurra a nota para insatisfeito. A hiperbole em -0,65 e a evidência de que o cruzamento do corpus (seção acima) funcionou — sem ele ela teria aprendido o sinal contrário.

O que continua valendo da versão anterior deste passo: as guardas de vazamento medem o SIMULADOR e são sobre a estrutura, não sobre o texto do B2W. `incongruencia_hiperbole` melhorou mas não chegou aos três: depois da correção ela dispara em DOIS dos três rótulos — `insatisfeito` 34/60 e `satisfeito` 39/60 (números medidos acima, na seção do retreino) —, e `neutro` continua em 0/60. Isso é limitação declarada, não vazamento remanescente: fala neutra raramente intensifica termo polar com hiperbole, e a decisão foi não forçar uma frase artificial no gerador só para a classe deixar de ficar muda. A guarda

`test_nenhuma_feature_e_previsor_unilateral` (`tests/test_simulador.py`) tolera isso por construção — ela só falha quando a feature dispara em EXATAMENTE UM rótulo, então 2-de-3 passa verde. Isso é posição aceita sobre o corpus sintético atual, não descuido: fica registrado aqui para o próximo leitor não achar que a guarda promete "sinal nos três rótulos" quando ela só promete "não previsor unilateral de UM rótulo só". A guarda não foi alterada. 5. Copiar `fusor.joblib` e `importancias.json` para `modelos/` na raiz do repositório local (guardando o artefato anterior como `.bak-<N>`, onde N é o número de features dele) e então, nesta ordem: `uv run python scripts/conferir_fusor.py` e `uv run pytest -q` inteiro. O laudo faz automaticamente os passos 3 e 4 desta lista — ver "Conferência do artefato" abaixo.

5.3.7 A ironia sai do vetor (04/09/2026)

Medição sobre o próprio corpus de treino do fusor (B2W-Reviews01, amostra equilibrada de 500 resenhas por rótulo, semente 20260904), rodando só a cabeça de ironia:

```
resenha NEGATIVA (nao): media P(ironia) = 0,1012  mediana 0,0013  frac(P>0,5) = 0,092
resenha POSITIVA (sim): media P(ironia) = 0,7146  mediana 0,9687  frac(P>0,5) = 0,740
                        diferenca (sim-nao) = +0,6134
```

A cabeça, treinada no IDPT 2021 (tweet e notícia), aplicada a resenha de e-commerce funciona na prática como **detector de sentimento positivo**: marca 74% das resenhas satisfeitas como ironicas contra 9% das insatisfeitas. O fusor de 40 features aprendeu **+0,77** de peso para `ironia_prob_media` no eixo satisfeito-menos-insatisfeito — mais ironia empurrando para SATISFEITO, o inverso do que o nome da feature promete. O fusor de 35 já tinha o mesmo sinal (+0,48), então não é regressão desta rodada: é o mesmo vazamento medido de novo, agora com número.

Consequência observável: a frase canônica de ironia deste projeto — "que atendimento maravilhoso, só esperei 3 horas" — pontuava **99,98 / nota 10 / promotor** no fusor de 40, porque as duas features de ironia empurravam para cima em vez de para baixo. (A cabeça isolada acerta essa frase: 0,998 nela, 0,002 num elogio sincero. Quem lia ao contrário era o fusor, e só por causa do corpus.)

Decisão: `ironia_prob_media` e `ironia_prob_max` saem de `NOMES_FEATURES`. Feature que mede outra coisa que não o nome dela é pior que feature ausente — ela não adiciona sinal, duplica `texto_prob_satisfeito_*` com ruído, e inverte o caso que o projeto usa como exemplo de manual. A cabeça de ironia CONTINUA carregada e obrigatória (invariante 7) e continua sendo lida por mensagem e exibida na dashboard, onde é honesta e já vem com a ressalva de confiabilidade que a tela mostra — o que muda é que ela para de pontuar. O retreino com 38 features foi executado no mesmo dia; ver a seção seguinte.

Nota sobre a janela entre a mudança do contrato e o artefato novo. Enquanto `modelos/fusor.joblib` era o artefato de 40 e o contrato já estava em 38, a API **nao subia** — `Fusor.carregar` passou a comparar `n_features_in_` com `len(NOMES_FEATURES)` e

levantar `FusorIncompatibleError` (PR #29, 04/09/2026). Antes dessa validacao, o mesmo desencontro deixava a API subir normalmente e estourar HTTP 500 so na primeira pontuacao real — falha tardia, no pior momento possivel. A validacao troca isso por falha alta e explicita na subida, que e o que a invariante 7 pede.

5.3.8 O fusor de 38 features — 04/09/2026

Notebook 02 rodado de novo com a ironia fora do vetor. **Acuracia 0,9433** (f1_macro 0,9436), contra 0,9467 do fusor de 40. A queda de ~0,003 e o resultado saudavel e esperado: as duas features removidas *estavam* ajudando o classificador — so que como detectores de sentimento positivo duplicando `texto_prob_satisfeito_*`, nao como medida de ironia. Perder um pouco de acuracia trocando uma feature que mente por nenhuma feature e o negocio certo.

Conferido com `uv run python scripts/conferir_fusor.py` (ver secao "Conferencia do artefato" abaixo): contrato 38/38, `classes_ = [0, 1, 2]` (invariante 8), nenhuma feature com peso perto de zero, e as cinco de incongruencia mantiveram a direcao negativa que a literatura preve:

feature	fusor de 40	fusor de 38
<code>incongruencia_emoji_texto</code>	-0,95	-1,03
<code>incongruencia_hiperbole</code>	-0,65	-0,64
<code>incongruencia_polaridade</code>	-0,37	-0,37
<code>incongruencia_marcador_contraste</code>	-0,38	-0,27
<code>incongruencia_aspas_ironicas</code>	-0,15	-0,19

Tres pesos com sinal aparentemente invertido, e por que NAO sao vazamento. O laudo sinaliza `texto_prob_satisfeito_ultima` (-0,50), `emoji_frac_positivos` (-0,18) e `emoji_frac_negativos` (+0,60) como suspeitos, porque o nome promete o sinal contrario. Conferidos contra os artefatos de 40 e de 35, os tres ja estavam assim nos dois — nao e efeito da remocao da ironia. A explicacao e **colinearidade**: `texto_prob_satisfeito_medio` pesa +2,81 e `emoji_score_medio` +1,56, e features fortemente correlacionadas com elas recebem coeficiente negativo de correcao na regressao logistica. E o comportamento normal do modelo com features redundantes, nao inversao de semantica como a de `ironia_prob_medio`. A diferenca entre os dois casos e verificavel: a ironia foi medida diretamente no corpus e provou medir outra coisa; estas tres continuam medindo o que o nome diz, so nao carregam informacao independente das dominantes. Fica registrado aqui porque o laudo vai sinalizar as tres em todo retreino futuro, e o proximo leitor precisa saber que ja foram investigadas.

O que este retreino NAO resolveu. A frase canonica do projeto continua pontuando alto:

"que atendimento maravilhoso, so esperei 3 horas"	score 99,95	nota 10
"otimo, mais uma vez ninguem resolveu nada"	score 99,74	nota 10
"resolveu rapido, muito obrigado! adorei"	score 97,82	nota 10

As cinco features de incongruencia dao **0,0 nas duas frases ironicas** e disparam na satisfeita de verdade (`polaridade` 0,5, `hiperbole` 0,5) — ou seja, hoje elas pegam entusiasmo genuino e nao pegam ironia. Nao e regressao: e o limite estrutural das cinco, todas funcao de `anotar_texto/emoji`, e "so esperei 3 horas" nao tem nenhuma palavra polar no SentiLex — e negativo por pragmatica. O diagnostico completo e a rota proposta (`incongruencia_situacao_negativa`, que levaria o vetor a 39 e exige mais um retreino) estao em `docs/superpowers/specs/2026-09-04-incongruencia-implicita-design.md`.

Corrigindo a previsao registrada na secao anterior: esperava-se que a frase deixasse de ser empurrada para cima com a saida da ironia. Nao foi o que aconteceu — o `texto_prob_satisfeito_medio` da propria cabeca de texto e 0,858 nela, e domina o eixo sozinho. A remocao consertou o vetor; nao tocou nesta frase.

5.3.9 O contrato sobe para 39: incongruencia implicita (04/09/2026)

`incongruencia_situacao_negativa` entrou em `NOMES_FEATURES` -- elogio convivendo com situacao negativa de atendimento na mesma fala. E a unica das seis que alcanca a frase canonica do projeto, porque nao depende de um segundo termo polar no lexicon.

O retreino do notebook 02 e OBRIGATORIO antes de a API voltar a subir. O artefato de 38 nao carrega mais:

`Fusor.carregar` valida a dimensao e levanta `FusorIncompativelError`. Isso e o comportamento desejado, nao um bug.

Medido ANTES de aceitar, no B2W-Reviews01 (2000 resenhas por rotulo, semente 20260904) -- que e o corpus que da o TEXTO do treino:

```
insatisfeito 82/2000 = 4,10%
neutro       53/2000 = 2,65%
satisfeito   35/2000 = 1,75%
```

Dispara nos tres rotulos, gradiente suave, razao 2,3:1 entre os extremos. Compare com a cabeca de ironia que saiu do vetor no mesmo dia (74% contra 9%, razao 8:1): aquilo era disjuncao pratica, isto e sinal.

Uma armadilha de metodo que quase repeti. Medir esta feature no corpus do SIMULADOR dava zero nos tres rotulos, e eu quase concluí que ela nasceria com peso zero. Errado, e pelo mesmo motivo registrado na correcao de 04/09 mais acima:

`FRASES_POR_ROTULO` **nao treina o fusor**. O notebook 02 usa 400 frases do B2W por classe e pega do simulador so a ESTRUTURA da conversa. O simulador importa para a guarda `test_nenhuma_feature_e_previsor_unilateral`, nao para o peso aprendido. As frases dos tres rotulos foram cruzadas mesmo assim -- sem isso a feature ficava constante em zero la e a guarda passaria por vacuidade, que e o modo de falha silencioso dela.

5.3.10 Conferencia do artefato

```
uv run python scripts/conferir_fusor.py          # laudo completo
uv run python scripts/conferir_fusor.py --sem-sonda # pula as frases-sonda
```

Laudo do artefato vigente em `modelos/`, em seis itens: contrato, ordem das classes, sinal dos pesos por familia, pesos perto de zero, tres frases-sonda e as metricas do treino. Instrumento, nao botao — nao escreve nada, no modelo de `scripts/medir_faixas.py`.

Ele existe porque **acuracia nao denuncia** os dois defeitos que esta semana custaram dois retreinos: o peso invertido de `ironia_prob_media` e o vazamento de `incongruencia_hiperbole`. Os dois so apareceram porque alguem olhou o sinal dos coeficientes e a distribuicao por rotulo — exatamente o que uma pessoa cansada depois de um treino longo pula. Por isso a acuracia nunca e impressa sozinha no laudo: vem sempre com o lembrete da invariante 10, e com alerta explicito acima de 0,97.

5.4 Contrato de features

`NOMES_FEATURES`, em `fraus/fusor.py`, e a lista canonica. `vetorizar` levanta `KeyError` se faltar chave — nunca zero silencioso (invariante 9).

Hoje sao **39 features**, de SETE familias no vetor (OITO sinais existem no sistema — a ironia continua existindo e sendo lida por mensagem, so nao entra mais aqui, ver a secao acima). O contrato subiu de 16 para 35 em 21/08/2026, quando os notebooks 03 e 04 passaram a existir e a condicao que justificava a espera acabou; de 35 para 40 em 03/09/2026, quando a familia

`incongruencia_*` entrou (ver [Retreino do fusor apos as features de incongruencia](#) acima); e caiu de 40 para 38 em 04/09/2026, quando `ironia_prob_media` e `ironia_prob_max` saíram (ver [A ironia sai do vetor](#) acima).

senal	modulo	features	no vetor?
texto	<code>fraus/sinais/texto.py</code>	4	sim
emoji	<code>fraus/sinais/emoji.py</code>	5	sim
tempo	<code>fraus/sinais/tempo.py</code>	7	sim
emocao	<code>fraus/sinais/emocao.py</code>	8	sim
lexico	<code>fraus/sinais/lexico.py</code>	3	sim
ironia	<code>fraus/sinais/ironia.py</code>	2	nao (lida por mensagem, nao pontua)
estilo	<code>fraus/sinais/estilo.py</code>	6	sim
incongruencia	<code>fraus/sinais/incongruencia.py</code>	5	sim

Consequencia: `montar_features` exige DOIS classificadores (texto e emocao); `Motor` continua exigindo os TRES (a ironia entra na atribuicao por mensagem, nao no vetor), e a API nao sobe sem os tres artefatos em `modelos/`. E o comportamento correto da invariante 7.

5.5 Notebook 03 — classificador de emocao

Requisito de banca: distinguir as **sete emocoes humanas**. A cabeca treinada tem sete classes, e a setima emocao e derivada — ver abaixo.

A ordem das classes **nao e redigitada no notebook**: ele importa `NOMES_EMOCoes` de `fraus.sinais.emocao`, a mesma lista que o sinal usa para ler as probabilidades. Redigitar seria o jeito mais facil de treinar `raiva` no indice 2 e le-la no indice 3.

5.5.1 Ordem canonica das classes

`fraus/sinais/emocao.py` fixa a ordem, e ela vale no notebook, no sinal e no fusor. Inverter nao gera erro: atribui a emocao errada em silencio, a mesma armadilha da invariante 8.

indice	classe
0	alegria
1	tristeza
2	raiva
3	medo
4	nojo
5	surpresa
6	neutro

5.5.2 Corpus: go_emotions_ptbr

Traducao PT-BR do GoEmotions (Demszky et al., 2020 — 58k comentarios do Reddit, 27 emocoes + neutro), com 43.410 exemplos de treino, 5.426 de validacao e 5.427 de teste. Licenca Apache 2.0. As 27 categorias sao reduzidas as seis de Ekman pelo **mapeamento oficial que o proprio GoEmotions publica** — nao por agrupamento inventado aqui.

O arquivo vem no formato CRU do GoEmotions — **uma linha por anotador**, nao por exemplo. Um rotulo conta quando **pelo menos 2 anotadores** o marcaram (a maioria dos exemplos tem 3 anotadores, entao 2 e maioria simples), e exemplos que caem em mais de uma classe de Ekman sao descartados: a tarefa e multiclasse de rotulo unico, e forcar vencedor entre duas emocoes empatadas inventaria rotulo.

O limiar mais alto rende MAIS exemplos, o que so parece contra-intuitivo ate ver o motivo — com 1 anotador quase todo exemplo recebe varios rotulos Ekman e cai fora do filtro de rotulo unico:

limiar	exemplos com rotulo unico
1 anotador	17.367
2 anotadores	46.014

5.5.3 O desequilibrio, e por que NAO subamostrar

Distribuicao medida com limiar 2:

classe	exemplos
alegria	19.284
neutro	13.603
raiva	5.155
surpresa	4.339
tristeza	2.601
medo	588
nojo	444

Sao **43:1** entre a maior e a menor. Os notebooks 01 e 02 balanceiam por subamostragem, mas aqui isso seria destrutivo: cortar tudo ao tamanho de `nojo` deixaria **3.108 exemplos** para sete classes, jogando fora 93% do corpus.

Escolha: pesos de classe na perda, pela formula $n / (k * n_c)$ — a mesma do `class_weight='balanced'` do scikit-learn. Os 46 mil exemplos ficam. Por isso a metrica de selecao do melhor checkpoint e o **F1-macro, nunca a acuracia**: com 43:1, um modelo que chutasse `alegria` sempre marcaria 42% sem ter aprendido nada.

5.5.4 O teto do desprezo — a declarar no relatorio

`nojo` e a classe mais rara do corpus, e o **desprezo e derivado de raiva x nojo**. A qualidade do desprezo fica limitada de cima pela qualidade da pior classe do modelo: se o F1 de `nojo` sair baixo, o desprezo herda isso. O notebook imprime esse teto explicitamente e grava o F1 por classe em `metricas_emocao.json`, para o numero aparecer no relatorio em vez de ser descoberto pela banca.

Limitacao a declarar: a traducao foi feita por maquina (Google Tradutor), sem revisao humana. Por isso a avaliacao reporta tambem o **XED-pt** (7.220 linhas, CC-BY, Plutchik 8 + neutro) como conjunto de teste independente, que nao passou por traducao automatica. O mapeamento Plutchik -> Ekman descarta *anticipation* e *trust*; o resto casa 1:1.

Duas ressalvas sobre o XED-pt: ele e **portugues europeu** ("pa", "artola"), enquanto o treino e PT-BR — diferenca de variedade, nao so de dominio; e o numero dele vai sair **mais baixo** que o interno, o que e esperado e e justamente o ponto, porque e ele que responde "mas nao e so traducao automatica?".

5.5.5 Por que o desprezo NAO e treinado

A setima emocao e o **desprezo**, e nenhum corpus rotulado em portugues o anota. Havia um corpus multilingue sintetico com `contempt` entre os rotulos, e ele foi **recusado de proposito**: treinar seis classes com dado humano e so o desprezo com dado de outra procedencia ensinaria o modelo a reconhecer o *estilo do texto sintetico* em vez do desprezo. E o mesmo vazamento da latencia disjunta, com outra roupa.

O desprezo e derivado da **diade primaria raiva + nojo**, que e como Plutchik (1980) o define — referencia citavel, nao rotulo inventado. A composicao usa media **geometrica**, nao aritmetica: desprezo exige as duas emocoes juntas, e a aritmetica devolveria 0,5 para raiva pura sem nojo nenhum, o que e raiva e nao desprezo.

5.6 Notebook 04 — classificador de ironia

Classificacao binaria: 0 nao-ironico, 1 ironico.

Corpus: IDPT 2021, a tarefa de *Irony Detection in Portuguese* do IberLEF — 15,2k tweets e 18,4k noticias anotados.

5.6.1 O corpus NAO tem download aberto

Diferente dos outros notebooks, o 04 **nao baixa o corpus sozinho**: o IDPT 2021 nao esta publicado para download livre — nao ha copia no GitHub nem no Hugging Face, e a pagina da tarefa nao expoe link direto. E preciso **solicitar aos organizadores** em <https://sites.google.com/inf.ufpel.edu.br/idpt2021/>.

O notebook aceita **tres origens** para o corpus, escolhidas na variavel `ORIGEM` da celula 3. Todas alimentam a mesma pasta e a mesma deteccao de esquema, entao trocar de fonte nao muda o resto do notebook:

<code>ORIGEM</code>	Quando usar
<code>sintetico</code> (padrao)	gera o corpus com <code>fraus.ingest.gerador_ironia</code> — nao depende de liberacao de ninguem
<code>drive</code>	os arquivos ja estao em <code>DIR_CORPUS</code> — foi assim que o IDPT entrou, se liberado
<code>kaggle</code>	baixa da conta Kaggle; exige <code>KAGGLE_USERNAME</code> / <code>KAGGLE_KEY</code> nos Secrets do Colab
<code>upload</code>	seletor de arquivos do navegador, para corpus que chegou por e-mail

5.6.2 Alternativas ao IDPT — levantamento de 14/08/2026

Registrado para ninguém repetir a busca. O que **nao serve**, e por que:

Fonte	Veredito
<code>arbml/multilingual_irony</code> (HF)	so <code>tweet_id + label</code> , sem texto ; reidratar exige API paga do Twitter
<code>ramondomingos/ptbr-irony-idioms-regionalism</code> (HF)	< 1.000 linhas, e e <i>benchmark de LLM</i> com gabarito em prosa — nao e corpus de classificacao
<code>rafaelanchieta/PiLN</code> (GitHub)	sao os modelos treinados do IDPT, nao o corpus; embeddings dependem de servidor antigo do NILC
IroSvA / SemEval-2018 traduzidos	desaconselhado : traducao automatica reintroduz o vazamento de procedencia que ja custou o primeiro fusor

O candidato mais citado e o corpus de **Goncalves et al., BraSNAM 2015** ("Bazinga! Caracterizando e Detectando Sarcasmo e Ironia no Twitter", UFMG), que circula no Kaggle. Lendo o artigo: sao ~2.628 tweets por classe coletados pelas hashtags `#sarcasm` e `#irony` e caracterizados com LIWC — ou seja, e um trabalho brasileiro sobre o Twitter **anglofono**. Corpus em ingles. Usa-lo aqui seria o vazamento de procedencia com selo academico.

Verificado tambem e descartado: `OpenNeuro ds004533` ("Ironia VEV") e **neuroimagem** — ressonancia de 40 sujeitos ouvindo frases ironicas, sem nenhuma frase rotulada para treino.

5.6.3 Onde existe corpus PT-BR de ironia (e como pedir)

Dois, os dois **sem download publico**:

- Vieira e Silva, A. (2025)** — *Deteccao automatica de ironia por meio de representacoes contextuais*, tese de doutorado em Linguistica, USP, orientacao de Marcos Lopes. Produziu um corpus de ironia em portugues com conversas da rede X: **1.186 exemplos** finais (de 1.200), anotados por **tres voluntarios humanos**, com o contexto da conversa. Achado relevante para o relatorio: os anotadores precisaram do **contexto de producao em ~30% dos casos** — evidencia direta de que ironia em mensagem isolada tem teto baixo, e ela vale citada mesmo sem o corpus em maos.
- Projeto IDPT / UFPel** — <https://institucional.ufpel.edu.br/projetos/id/u3345>, coordenacao de **Larissa Astrogildo de Freitas**. E o grupo por tras do IDPT 2021. Enderece o pedido a coordenacao.

Tamanho importa aqui: 1.186 exemplos anotados a mao nao treinam um BERT do zero, mas sao excelentes como **conjunto de teste independente** — exatamente o papel que o XED-pt cumpre no notebook 03. E a arquitetura de honestidade que o projeto ja usa: treinar no que ha em volume, medir no que ha em qualidade.

5.6.4 O padrao: corpus sintetico

Enquanto nenhum dos dois chega, o notebook gera o proprio corpus com `fraus.ingest.gerador_ironia`. E o mesmo impasse do sinal de tempo e a mesma saida ja aceita: o fusor treina em conversas sinteticas porque nenhum corpus de review tem timestamps.

O gerador foi escrito **contra** o vazamento que matou o primeiro fusor. A armadilha obvia seria "ironico = elogio + fato ruim", que ensinaria o modelo a procurar fato ruim em vez da incongruencia. Tres cruzamentos impedem isso, cada um travado por teste em `tests/test_gerador_ironia.py`:

- os mesmos **fatos ruins** aparecem nas reclamacoes diretas (nao-ironicas);
- as mesmas **palavras elogiosas** aparecem em elogios sinceros e em reclamacoes que comecam agradecendo;
- numeros e unidades de tempo** aparecem nas duas classes;
- ha uma familia de ironia **por atenuacao**, sem palavra elogiosa nenhuma, para que "tem elogio" nao seja condicao necessaria.

A metrica interna sera otimista por construcao — ela mede o quanto o modelo aprendeu estes padroes, nao o quanto le ironia humana. Isso precisa estar no relatorio.

Por isso o `metricas_ironia.json` grava a **procedencia derivada de ORIGEM**, nunca `IDPT 2021` fixo: artefato que mente sobre a propria fonte e pior que artefato ausente.

Como o notebook **nunca viu os arquivos**, a celula de carga tem esquema **configuravel**: ela le todo `.csv / .tsv` da pasta, imprime as colunas encontradas, tenta achar a de texto e a de rotulo pelos nomes mais provaveis e **para nomeando as colunas disponiveis** se nao achar. Rotulo fora do mapeamento tambem para com erro, em vez de virar 0 silenciosamente.

Se o corpus nao for liberado a tempo, a saida honesta e declarar a ironia como trabalho futuro no relatorio. Trocar por um corpus de sarcasmo em ingles traduzido repetiria, com outro nome, o vazamento de procedencia que ja custou o primeiro fusor.

A metrica de selecao do melhor checkpoint e o **F1 da classe ironica isolada**, nao a macro nem a acuracia: e essa classe que o sinal consome, e corpus de ironia costuma ser desequilibrado o bastante para um modelo que responde "nao e ironia" sempre marcar boa acuracia. O treino tambem usa pesos de classe, pelo mesmo motivo.

Por que uma cabeca separada e nao mais uma classe do sinal de texto: ironia nao e um sentimento, e uma relacao entre o que o texto DIZ e o que ele SIGNIFICA. "Que atendimento maravilhoso, so esperei 3 horas" e lexicamente positivo e pragmaticamente negativo ao mesmo tempo. Como quarta classe de satisfacao, o modelo seria obrigado a escolher uma das duas leituras — e a informacao de que ha conflito, que e justamente o sinal de ironia, se perderia.

Limitacao a declarar: o IDPT e anotado em tweets e comentarios de noticia, nao em atendimento de chatbot. A transferencia de dominio nao esta verificada — ironia em reclamacao de suporte pode ter forma diferente de ironia em comentario de politica.

5.7 Sinal lexico — SentiLex-PT02

Nao tem notebook: e recurso pronto, nao treinado.

Fonte: SentiLex-PT02 — Silva, Carvalho e Sarmento, "*Building a Sentiment Lexicon for Social Judgement Mining*" (PROPOR 2012), CC-BY. 82.347 formas flexionadas na origem, **79.189** depois da conversao.

Preparo: `scripts/preparar_sentilex.py` converte o arquivo original em `fraus/dados/sentilex_pt02.csv` e imprime o relatorio do que mexeu — 315 entradas cuja polaridade veio do complemento (`POL:N1`), 9 com sinal normalizado (marcadas `REV:POL` na fonte, com o contador vazado no campo de polaridade) e 2.032 formas homografias com polaridades conflitantes, que foram **zeradas** em vez de decididas pela ordem do arquivo.

Por que um lexicon se o BERTimbau ja le o texto: porque ele nao foi treinado nos mesmos dados nem carrega os mesmos vieses. Dois sinais de procedencia independente que concordam sustentam a nota melhor que um sozinho — e a **discordancia** entre eles e o formato tipico da ironia.

Negacao por escopo: "nao foi otimo" inverte a polaridade de "otimo", com alcance de tres tokens, e o escopo **morre na pontuacao forte** — em "nao chegou. otimo atendimento" o *otimo* continua positivo.

Acento: o SentiLex e acentuado e cliente de chat nem sempre. Ha indice de reserva sem acento; das 74.443 chaves resultantes, apenas **24** colidem (`incomodo / incomodo`, `ingenua / ingenua`) e essas ficam zeradas — chute de polaridade errado e pior que termo ausente.

Limitacao do recurso, a declarar: o SentiLex e lexicon de **julgamento social** — anota polaridade dirigida a entidades humanas. E forte no adjetivo que julga (`pessimo`, `otimo`, `incompetente`) e **neutro em verbo de afeto do proprio falante**: `gostar`, `adorar` e `odiar` valem 0 nele. Por isso o sinal lexico complementa o BERTimbau e nao o substitui — quem le "adorei o produto" e o transformer.

5.8 Sinal de estilo

Nao tem notebook: e **deterministico**, nao treinado — `fraus/sinais/estilo.py` mede a FORMA da escrita (caixa alta, pontuacao enfatica, alongamento, palavra com gradacao, censura), nao o conteudo.

Por que determinístico e não um classificador. O BERTimbau não aprende ênfase porque o B2W-Reviews01 — resenha moderada de e-commerce — praticamente não contém gritaria nem xingamento. Modelo não aprende fenômeno que o corpus de treino não tem, e mais épocas sobre o mesmo texto só reproduziriam o mesmo artefato. Regra escrita à mão cobre exatamente o que falta: forma, não semântica.

Fonte do lexicon de palavras: `fraus/dados/palavroes_ptbr.csv`, curadoria própria, ~172 termos, colunas `termo, intensidade, alvo`. A intensidade é graduada (leve/medio/pesado, mapeada para 0,33/0,66/1,0) porque "que droga" e "vai tomar no cu" não são o mesmo evento, e o alvo (pessoa/desabafo) existe porque xingar o PRODUTO e reclamação enquanto xingar o ATENDENTE e ruptura da conversa. Nenhuma dependência externa entrou para isso, e o SentiLex-PT02 também não serve — ele anota polaridade de julgamento, não graduação de baixo calão nem alvo do xingamento.

Ordem censura-antes-de-normalização. `tem_censura` roda sobre a palavra CRUA, com a máscara (`*@#$$%`) ainda intacta, e só depois `normalizar` reverte homóglifo e apaga acento. Inverter a ordem apagaria a marca de censura antes de ela ser vista — "p*rra" perderia o asterisco e viraria uma palavra qualquer, nunca reconhecida como autocensura.

Casamento por curinga. `casar_censurado` trata cada símbolo de máscara como uma letra qualquer (regex com `.` no lugar do símbolo), não como uma letra fixa: `p*rra casa porra`, `c*ralho casa caralho`. Mapear o símbolo para uma vogal fixa acertaria um caso e erraria o outro em silêncio. Quando mais de um termo do lexicon casa o mesmo padrão com curinga, o desempate é pela MAIOR intensidade — determinístico, e não pela ordem em que a linha aparece no CSV, o que deixaria o resultado dependente de onde alguém inseriu a linha.

A execução do kkkk. Alongamento de caractere (`MINIMO_ALONGAMENTO = 3` repetições seguidas) é tratado como ênfase, exceto para as letras de riso (`k`, `h`): "kkkk" e o marcador POSITIVO mais comum do chat brasileiro, e contar risada junto de "naooooo" inverteria o sentido da feature em boa parte das conversas reais.

Limitação a declarar: o lexicon de palavras e curadoria própria, não recurso acadêmico publicado. Diferente do SentiLex-PT02 (Silva, Carvalho e Sarmento, PROPOR 2012) e do Emoji Sentiment Ranking usados pelos outros sinais, `palavroes_ptbr.csv` não tem paper citável nem revisão por pares — a cobertura e a graduação vieram de julgamento próprio, e isso precisa aparecer no relatório como o que é.

5.9 Notebook 05 — calibração, e por que ele não retreina nada

Acrescentado em 10/09/2026. Ele ajusta **um escalar por cabeça** (a temperatura) sobre os checkpoints que já existem, e mede o quanto isso aproxima a confiança declarada da frequência empírica — o **ECE**, mais o diagrama de confiabilidade.

Temperature scaling não muda a decisão do modelo. Dividir todos os logits pelo mesmo $T > 0$ preserva a ordem, então acurácia e F1 ficam idênticos. É a única forma de calibração que não pode piorar o classificador — e o notebook tem uma asserção que falha se o `argmax` mudar, porque isso seria defeito de código, não propriedade do método.

Ele produz `modelos/calibracao.json`. **Nada no runtime consome esse arquivo ainda, e isso é deliberado:** aplicar a temperatura muda o score de toda conversa do banco, e a régua deste projeto para trocar o que gera número é medir o efeito em `score/nota/categoria` antes de promover — a mesma disciplina que recusou a quantização int8 (ver `docs/encolhimento.md`).

A ironia é o caso a não confundir. Calibrar vai melhorar o ECE dela e ela vai continuar errando 6 em 10 falas sinceras de atendimento. Temperatura ajusta *o quanto* o modelo se diz confiante, não *sobre o que* ele está confiante. Apresentar a melhora do ECE como conserto do domínio seria falso.

5.10 Notebook 02 — a seção 10 é uma pendência, não uma etapa

A saturação das features de tempo está descrita lá com a medição, o conserto (`winsorizar no p99 + log1p` antes do scaler) e — o que mais importa — **o acoplamento**: a transformação precisa existir no notebook e no runtime, ou o score sai errado em silêncio. Um `fusor.joblib` treinado sem a compressão contra um runtime que comprime continua tendo `n_features_in_ = 39`, então `Fusor.carregar` **não pega** essa incompatibilidade.

5.10.1 As três saídas, medidas em 15/09/2026 — decidido: só log1p

O instrumento (hoje `scripts/retreinar_fusor_local.py`; a versão de medição está no commit `e719218`) reproduz este notebook localmente (mesmo B2W, mesmas sementes, mesmo `montar_features`, extração por ONNX) e **confere a reprodução antes de medir**: acurácia 0,9500 nos dois, score diferindo no máximo 0,0001 do `fusor.joblib` vigente. Nada foi promovido.

A coluna de espera é a frase canônica (texto 0,858 satisfeito, todo o resto na média do treino), com score, nota e categoria:

variante	acurácia	F1	categorias ≠ vigente (de 300)	60 s	300 s	600 s
vigente	0,9500	0,9501	—	99,9 · 10 P	98,1 · 10 P	44,5
só log1p	0,9467	0,9469	6	99,7 · 10 P	96,6 · 10 P	90,1
só teto p99	0,9433	0,9434	1	99,9 · 10 P	96,1 · 10 P	18,4
p99 + log1p	0,9467	0,9469	5	99,7 · 10 P	96,6 · 10 P	90,7

O que a tabela muda na recomendação escrita acima:

- **p99 + log1p conserta o relógio mandando sozinho apagando a espera longa.** O teto do treino é ~7 min (`latencia_mediana_s p99` = 424 s), então três horas pesam o mesmo que dez minutos: a frase canônica, que é **irônica**, sai promotor. Consertar um defeito criando o oposto não é conserto.
- **Só teto** não resolve nada: o penhasco só muda de lugar (10 min → nota 2).
- **Só log1p** é a única que mantém a espera **monótona e sem penhasco**: 10 min ainda promotor, 30 min detrator 6, 3 h detrator 1 — o texto modula, o relógio pesa cada vez mais devagar, e nenhum dos dois apaga o outro. Custa 0,33 ponto de acurácia no teste sintético.
- **Ablação da invariante 10**: só as quatro features de tempo acertam 0,513 (cru) e 0,527 (p99+log1p), contra 0,333 do acaso. O relógio carrega sinal do rótulo no simulador por construção, e nenhuma das variantes muda isso.

5.10.2 Promovido em 15/09/2026

As três partes indivisíveis, feitas juntas:

1. **Função única.** `log1p` mora em `fraus.fusor.vetorizar`, sobre `FEATURES_EM_LOG`. As chaves continuam em segundos (atribuição, tela e deriva leem segundos); só o vetor que o scaler recebe é comprimido. Treino, pontuação, contribuição e deriva passam pela mesma função.
2. **Artefato regerado** por `scripts/retreinar_fusor_local.py --promover`: acurácia 0,9467, F1-macro 0,9469. O anterior ficou em `modelos/fusor.joblib.bak-segundos`. O artefato agora carrega `fraus_escalas_do_tempo = "log1p"`, e `Fusor.carregar` **recusa** artefato sem essa marca: é o caso que `n_features_in_` não pega.
3. **Medição de categoria.** No teste do B2W, 27 notas e 6 de 300 categorias mudam. Nas 180 conversas do simulador, 23 notas e 7 categorias, com a distribuição indo de 60/63/57 para 60/60/60 (detrator/neutro/promotor).

`scripts/medir_dominio_do_tempo.py` contra o artefato novo: o relógio empata com o texto em **613 s** (antes 411 s), e com 3 h pesa **2,1x** o texto (antes 20x). Uma conversa sincera ("resolveu, muito obrigado") com 30 min de espera saía 0,30; agora sai 55,3, ainda detrator. Com 3 h, 21,95 em vez de 0,00.

Quem tem o artefato em outro lugar precisa regerar (a VM, o Drive): um `fusor.joblib` de antes de 15/09 não sobe mais, e isso é o comportamento correto da invariante 7.

6. Fraus — Design System

Contrato visual da interface. Quem escrever UI neste projeto segue este documento. Ele não descreve como as telas estão hoje: descreve como elas devem ser.

Mundo: *Pauta* — a conversa notada como partitura. **Chassi:** shadcn/ui sobre Tailwind v4, tokens em OKLCH, **dois temas, ambos escuros** — espaço profundo (padrão de fábrica) e chuva de neon (ver seção 8).

6.1 1. A tese, e por que ela governa a composição

Um atendimento é uma **sequência temporal de turnos com duração**. Isso é o que a notação musical resolve há quatrocentos anos, e resolve melhor do que a fileira de cartões que a categoria entrega.

A notação faz uma coisa que nenhum dashboard genérico faz: ela **separa o que está acima e o que está abaixo da linha**. Acima vai o articulado — letra, dinâmica, expressão. Abaixo vai o medido — cifra, andamento, baixo contínuo.

▮ **Acima da linha é o que foi DITO. Abaixo da linha é o que foi MEDIDO.**

Essa é a regra mestra da interface, e ela é **estrutural antes de ser cromática**. A metáfora de cor (âmbar/azul) permanece e passa a ter uma casa: âmbar mora acima, azul mora abaixo. Quando alguém precisa quebrar a regra de posição, a cor ainda desambigua — mas quebrar a posição é a exceção que se justifica no código.

Emenda de 04/09/2026 — `--linha` ganhou um segundo papel. Até aqui o token só fazia a régua do sistema, a linha que **afirma** dito/medido. Com a Task 6 (hierarquia), `CartaoIndicador` trocou `border-compasso` por `border-linha` como separador interno da armadura: `--compasso` foi desenhado mais fraco de propósito, para viver DENTRO de uma superfície de vidro, e a armadura deixou de ser painel — ela não tem vidro, chanfro nem borda —, então o filete que a separa por dentro precisa do peso de `--linha` para continuar visível sobre o papel nu. O token agora serve dois papéis — régua que afirma, e separador interno da armadura — porque o candidato mais fraco parou de bater no contexto novo; não é acúmulo por descuido.

6.1.1 1.1 O que é notação e o que é decoração

Nenhum glifo musical é desenhado. Sem clave, sem semínima, sem pentagrama de cinco linhas. A gramática entra como **estrutura e ritmo**, não como fantasia:

Elemento da notação	O que ele é aqui	Por quê
a linha	régua horizontal de referência que divide dito e medido	dá eixo comum a séries de escalas diferentes
a barra de compasso	filete vertical marcando a virada do dia	agrupa o tempo sem legenda
a pausa	intervalo proporcional à latência	silêncio com duração notada é exatamente a espera
a ligadura	marcação do trecho que puxou a nota	atribuição por sentença
a dinâmica	peso e tamanho do tipo, não cor	intensidade sem gastar canal de cor
a cabeça tracejada	atendimento medido com evidência fraca	o valor está na escala, e a linha que o afirma é interrompida
a cabeça vazada	atendimento sem sinal	ocupa o tempo e não soa: a regra "ausência não é zero" vira forma

A cabeça vazada é a peça mais importante desta lista. O princípio de produto "ausência de dado não é insatisfação" deixa de ser nota de rodapé e passa a ser **notação**: o marcador existe, ocupa a posição temporal, e é oco.

Se em algum momento ler esta interface exigir saber solfejo, a regra foi aplicada errado. O modo é **Operate**: expressão nunca obscurece a tarefa.

Emenda de 04/09/2026 — a cabeça vazada saiu do papel. Esta seção prometia a peça desde 13/08/2026 e a chamava de a mais importante da lista; ela levou até 04/09/2026 para existir de fato, em `components/CabecaVazada.tsx`: um anel oco (`rounded-full border border-muted-foreground bg-transparent`), sem preenchimento, com rótulo textual obrigatório ao lado (`comRotulo`, padrão `true` — a exceção documentada é quando quem chama já escreve um rótulo equivalente perto, para não duplicar leitura de tela).

O caso mais visível do vácuo era `EtiquetaCategoria: categoria: null` desenhava um ponto **cheio cinza** no mesmo slot dos três pontos cheios coloridos da escala (detrator/neutro/promotor) — exatamente "cinza dentro da escala", que a §5 proíbe por nome. Corrigido: `categoria: null` agora renderiza `<CabecaVazada comRotulo={false} />` (o rótulo por extenso já é escrito por `EtiquetaCategoria` logo depois do anel), então o ausente deixou de competir por posição com os três valores reais.

Emenda de 10/09/2026 — a terceira forma fecha o trio. Havia duas formas, cheia e vazada, e entre elas um caso sem notação: **tem dado e não há evidência suficiente para afirmar**. Uma conversa cujo cliente escreveu só "ok" saía **cheia**, com a mesma confiança visual de uma de 40 turnos — as duas tinham nota, só uma tinha material para sustentá-la.

forma	significado
cheia	medido, com evidência
tracejada	medido, evidência fraca
vazada	não medido

A tracejada **mantém a cor da classe** e troca o preenchimento por contorno interrompido (`border-dashed`, `bg-transparent`). Não vira cinza: cinza é fora da escala, e é o significado reservado à ausência — a categoria continua valendo, o que está em dúvida é quanta fala a sustenta. E não vira meio-preenchida: isso seria uma segunda leitura quantitativa competindo com a nota.

Ela vive dentro de `EtiquetaCategoria`, não num componente que quem chama escolhe. Esse é o **slot único da cabeça** — tabela, piores atendimentos, ficha do grafo e tela do atendimento passam todos por ele —, e três formas oferecidas em quatro lugares dariam uma que ninguém lembraria de trocar, e ela continuaria anunciando confiança que não existe.

O critério nunca é a probabilidade do modelo. Probabilidade não calibrada não é confiança, e apresentá-la como se fosse é exatamente a desonestidade que este produto recusa. A evidência é **observável** — quantas mensagens o cliente mandou, quantas palavras escreveu — derivada em `fraus/evidencia.py`, no servidor. Pelo mesmo motivo o campo se chama `evidencia_fraca` e **não** `confianca`.

Rótulo obrigatório, como na vazada — e aqui ele vai além de nomear o estado e imprime o **motivo**: "evidência fraca" não aciona ninguém, "menos de 5 palavras do cliente" aciona.

6.2.2. Composição

6.2.1 2.1 O que sai

- **A fileira de cartões-KPI no topo.** É o "template de métrica-herói", e ele empurra a tese da tela para baixo da dobra.
- **O cartão como estrutura de página.** Painel com borda e fundo próprio deixa de ser o agrupador padrão; agrupa-se por espaço e régua.

Emenda de 21/08/2026. O painel voltou a ter fundo e borda próprios ao adotar o vidro líquido (ver seção 7). A regra original continua valendo no que ela realmente defendia — a página é uma pilha de sistemas, não uma grade de cartões, e quem agrupa continua sendo espaço mais régua. O que mudou é que o sistema agora tem **material**, e material precisa de superfície. Registrado como custo assumido, não como regra revogada. - **O parágrafo explicativo acima de cada painel.** A prosa metodológica não some — muda de lugar (ver 4).

6.2.2.2.2 O que entra

O sistema é a unidade de composição. Como numa partitura, um *sistema* é uma faixa horizontal de largura total que carrega uma linha de tempo e tudo que a anota. A página é uma pilha de sistemas, não uma grade de cartões.

A armadura. Os indicadores agregados (NPS, CSAT, contenção, latência) vivem à esquerda do primeiro sistema, empilhados e compactos — como a armadura de clave, que se lê de uma vez e não se relê a cada compasso. Eles são **rótulo mais número tabular**, sem barra de progresso decorativa e sem cartão.

Ritmo vertical. Densidade varia entre sistemas: um sistema denso ganha o direito de um respiro depois. Mais espaço acima de um título do que abaixo.

Emenda de 04/09/2026 — a armadura deixou de ser painel, e ganhou nível. Esta seção já pedia isso ("sem cartão", "rótulo mais número tabular"), e até aqui a implementação não cumpria. `Painel` ganhou a prop `nível` (`"dominante"` | `"apoio"`, padrão `apoio` — um painel que não declara nível não pode virar dominante por omissão, senão a tela passa a ter dois dominantes e nenhuma resposta principal), e o vocabulário — as duas classes de espessura de vidro que cada nível usa — mora em `lib/hierarquia.ts`, pelo mesmo motivo que cor mora em token: regra digitada de novo em outro lugar diverge.

O mesmo arquivo expõe `DOMINANTE_POR_TELA`, um mapa de rota para o **identificador estável** do sistema que responde à pergunta que levou o analista àquela tela — não o título de exibição. A distinção importa: vários painéis montam o título em tempo de render, porque ele carrega período ou contagem (`Atendimentos de ${rotulo}, ${n} nós, ${m} arestas`), e nenhuma string estática casaria com isso. Quem marca o dominante de fato é a própria tela, com `nível="dominante"` no `Painel`; o mapa é documentação da intenção e uma referência para conferir "um por tela", não chave de busca.

6.3 3. Tokens

Tokens em `app/globals.css`; consumo via classes utilitárias. Nada de hex solto em componente.

Os valores abaixo são os do tema **grafite**, o padrão de fábrica. O tema chuva de neon sobrepõe apenas o chassi e não encosta na camada de dado — ver seção 8.

6.3.1 3.1 O fundo é papel, não vazio

O fundo não é preto: é **papel de ensaio grafite**. Preto puro faz a régua flutuar no vácuo e transforma dado em néon. A superfície precisa parecer estoque de papel sob luz de escritório — o analista fica horas nela.

Emenda de 24/08/2026 — o papel é pautado. A regra acima dizia papel liso. O fundo passou a carregar **linhas horizontais de ritmo constante**, nos dois temas, por decisão do dono do projeto.

O que **não** mudou, e é o que a regra original defendia de verdade: o fundo continua sendo papel e não vazio, continua sem preto puro, e continua sem competir com o dado — as linhas moram no ateliê, a camada que não carrega informação, e passam **por baixo** da luz.

Elas não são um pentagrama. A §1.1 proíbe desenhar glifo musical e manda a gramática entrar como estrutura e ritmo; agrupar em cincos seria desenhar a pauta, ritmo constante é papel pautado. A régua do sistema (`--linha`) segue sendo a única linha que **afirma** alguma coisa.

O custo veio na hora e está pago: a textura levantou o pior caso das superfícies translúcidas nos dois temas. Os pisos foram **remedidos** (o grafite saiu de 0,25/0,24/0,23 para 0,309/0,28/0,246) e o vidro fino da chuva precisou engrossar de novo — 68% → 71% —, porque `--destructive` sobre ele havia caído para 4,46:1. A ordem de sempre: a espessura cede, o portão não.

Emenda de 24/08/2026 (segunda) — a marca é impressa no papel, e o papel ficou forte. A emenda anterior autorizou o papel pautado, mas a implementação entregou grafite a 5,5% de opacidade: 1px a cada 13px, a 5,5%, sobre fundo escuro, **não chega a ser visto**. A peça existia no código e não na tela. A contenção já tinha sido negociada e perdida; foi a implementação que continuou protegendo o contrato antigo. Corrigido: **0,055 → 0,12** no grafite e **0,13 → 0,22** na chuva.

O fundo passou também a carregar o **monograma da marca**, sangrando pela quina inferior direita, nos dois temas.

É máscara, não imagem de fundo, e a distinção é o inteiro da peça. O arquivo da logo **carrega um fundo opaco** — o PNG era RGB sem canal alfa, e o SVG que o substituiu traz um `<rect>` preto cobrindo os 640×640. Usar qualquer um dos dois como `background-image` pintaria um quadrado opaco por cima do ateliê, que é exatamente o bug que o `SidebarInset` com `bg-background` já causou. `public/fraus-marca.svg` é o mesmo desenho **sem esse fundo** e com as duas letras forçadas a branco: quem pinta é o `background` do elemento, via `--marca-cor`. É isso que entrega "a logo com o tema respectivo aplicado" — dourada no grafite, magenta na chuva — sem gerar dois arquivos e sem tema novo pedir asset novo.

Emenda de 25/08/2026 — a fenda, e a rampa em token. O monograma ganhou uma diagonal (`M110,470L560,180`, 18 unidades de espessura) atravessando o `F` e o `S`. Aos 28 px da navegação ela dá 0,8 px: parada, lê como detalhe de desenho. Ela é a **junta** por onde a marca se abre no gesto dos cinco cliques — sem ela a revelação seria efeito colado por cima do desenho, e não o desenho se abrindo pela própria costura. **Os dois arquivos a recebem**, para continuarem sendo o mesmo desenho; na marca do ateliê ela é recorte de alfa.

A rampa do gradiente virou dois tokens, `--marca-cor-pe` e `--marca-cor-topo`, e **isso obrigou o desenho a virar SVG inline** em `MarcaFraus.tsx`: um SVG servido por `<img>` é documento externo, onde `var(--...)` não atravessa a fronteira e animar uma parte não existe. `public/fraus-logo.svg` sobrevive com o dourado literal para o `og:image` e o README, que consomem a marca fora do React, onde token nenhum chega. A duplicação é o preço de a marca existir dentro e fora da aplicação, e fica registrada aqui para não ser "consertada" por engano.

`--marca-cor` (chapada) e a rampa (duas paradas) são coisas **diferentes**: a primeira pinta a marca impressa no fundo do ateliê, que é máscara e não tem gradiente. Trocar uma pela outra apaga o gradiente que é a marca.

Emenda de 25/08/2026 (segunda) — a única mentira autorizada da interface. Enquanto o painel dos cinco cliques está aberto, os quatro indicadores da visão geral exibem números **falsos**. É violação deliberada e **encenada** da §5 ("sem sinal é cabeça vazada, nunca zero") e da invariante 2, e ela só se sustenta porque encena. As quatro travas, em `lib/mentira.tsx`:

1. **Nada sai da memória** — zero escrita no banco, zero chamada à API. É estado de render, e morre no fechamento. Nenhum agregado, export ou ordenação chega perto disto.
2. **Leitor de tela recebe a verdade** — o número falso é `aria-hidden`, e um `sr-only` ao lado diz que o valor foi falsificado e qual é o real.
3. **O selo é inescapável** — o aviso em `--destructive` fica no mesmo quadro dos números. Não existe captura de tela da dashboard mentindo sem ele dentro dela.
4. **Volta sozinha** — 8 s, `Esc` ou clique fora, e também no desmonte do componente (quem navega com o painel aberto não deixa a mentira para trás).

Gráficos e tabela não mentem, por decisão: espalhar isso pelo Recharts é onde deixa de ser contido e passa a arriscar sobrar na tela depois do fechamento — o defeito que as travas existem para impedir. O trilho de referência **some** enquanto o número mente: o ponteiro marca a posição verdadeira, e ao lado de um número falso ele leria como bug de render, não como ironia.

O "fade + gradiente" é uma **segunda camada de máscara** cortada por `mask-composite: intersect`: a marca nasce sólida onde sai da tela e se dissolve subindo. Sem isso ela leria como adesivo colado.

O que **não** mudou: nada disso carrega dado. É a mesma camada do ateliê, o único lugar onde cor saturada não inventa canal de significado, e a §1.1 continua valendo — o monograma é a marca, não um glifo musical.

O custo, de novo pago na hora: `--muted-foreground` e `--destructive` sobre vidro fino caíram para 4,25 e 3,86. A espessura cedeu — 45% → 55% no fino e 62% → 72% no médio no grafite, 71% → 81% e 75% → 85% na chuva. Ver §8.6.

Emenda de 24/08/2026 (terceira) — o fundo ganha grade e sol. O fundo passou a carregar uma **grade em perspectiva** (linhas convergindo para um horizonte a 320px, espaçadas a 44px, `--grade-op` em 0,1 no grafite e 0,2 na chuva) e um **sol listrado** no horizonte (`--sol-op` em 0,16 no grafite e 0,3 na chuva), nos dois temas.

O que **não** mudou: as duas peças moram no ateliê, a mesma camada do papel pautado e da marca impressa, não carregam dado, e passam **por baixo** da luz — a mesma garantia das duas emendas acima, agora estendida a mais duas peças.

O custo, de novo em pisos e espessura de vidro: o vidro fino da chuva foi de 71% para **74%**, e o vidro médio da chuva **recuou** de 75% para 74% — ele havia subido para 81%/85% no meio do trabalho da Task 8 e voltou quando um bug de medição foi corrigido. O saldo desta sessão contra o início dela: fino subiu 3 pontos, médio caiu 1. No grafite, o vidro fino foi de 55% para **64%**; médio (72%) e denso (82%) da chuva (86%) ficaram como estavam.

Token	Papel
<code>--background</code>	estoque de papel; grafite neutro, nunca <code>#000</code>
<code>--card</code>	segunda camada de papel, para o que precisa de leve elevação
<code>--linha</code>	a régua do sistema; meio ponto, sempre visível e nunca dominante
<code>--compasso</code>	filete de virada de dia; mais fraco que <code>--linha</code>
<code>--foreground</code>	tinta
<code>--muted-foreground</code>	piso de cor de texto — nada mais claro que ele carrega texto

6.3.2 3.2 As duas vozes

Token	Voz	Onde mora
<code>--dito</code>	âmbar — fala, texto, emoji, o que o cliente articulou	acima da linha
<code>--medido</code>	azul — score, probabilidade, latência, tendência	abaixo da linha

Cada uma tem variante `--texto` com contraste verificado. Cor de marcação e cor de tipo não são a mesma coisa: `npm run contraste` é o juiz, e todo par que carrega texto cruza AA (4.5:1).

3.2.1 No grafo, as duas vozes viram duas famílias

O canvas do grafo da memória é a única tela com **nove tipos de nó**, e duas cores não distinguem nove coisas: `categoria`, `canal`, `fonte` e `feature` saíam todas do mesmo azul, e quem olhava não separava o eixo aprendido pelo fusor de um canal de atendimento.

A regra não foi trocada — ela subiu de nível. **A família continua dizendo dito/medido; o matiz dentro dela passou a dizer o tipo:**

Família	Faixa de matiz	Tipos
dito (quente)	30-95	emoji 30, conversa 60, termo 95
medido (frio)	150-340	fonte 150, canal 195, feature 265, categoria 305, desfecho 340

Tokens `--no-<tipo>`, todos em **L 0.75 / C 0.14**. Claridade e croma iguais são deliberados: matiz distingue o tipo, e claridade não pode virar hierarquia acidental — um nó mais claro pareceria mais importante, e nenhum tipo é mais importante que outro.

`--no-importacao` é a exceção, e ela é semântica: cinza acromático, porque importação é um nó **solto** cuja origem o schema não rastreia, e cinza é a ausência de afirmação — a mesma disciplina da cabeça vazada do `sem_sinal`.

A leitura de longe ("isto é fala ou é medida?") sobrevive intacta; a de perto ("que coisa é esta?") passa a existir. E porque essas nove cores só existem nesta tela, ela carrega **legenda própria** agrupada pelas duas famílias: nove matizes sem legenda seriam o grafo afirmando uma distinção que o leitor não tem como ler.

3.2.2 Na partitura da fala, cada emoção tem cor — e a ordem faz parte da cor

Pedido do dono do projeto em 15/09/2026. Emoção é leitura **medida**, e a pauta mostra classe e emoção juntas, então os tons obedecem a duas exclusões: nenhum perto das três classes (`--detrator 22`, `--neutro 90`, `--promotor 190`), senão "raiva" leria como "insatisfeito"; nenhum perto do âmbar do dito (60).

Nove cores num painel não se separam — medido, não suposto: com as três classes na conta, nenhum conjunto de 4 a 6 tons passou o validador em todos os pares. O que torna a paleta legítima são duas coisas juntas:

1. **Cada linha tem o nome escrito.** Cor nunca é a única identificação.
2. **A validação é por vizinhos, numa ordem fixa.** A ordem da pauta é `medo · nojo · tristeza · raiva · surpresa · alegria · neutro` — negativas agrupadas, neutro por último. Das 5040 ordens, 78 passam com os tons de `--emo-*`; a ordem do modelo não passa. Pares vizinhos: CVD ΔE 12,5, visão normal 16,2, as sete ≥ 3:1 sobre o card.

`--emo-neutro` é acromático pela mesma disciplina do `--no-importacao`: ausência de afeto. Desprezo, derivado de raiva + nojo, é a nota **vazada** no tom da raiva. A cor vai só na haste e na nota; o número segue em `foreground / muted-foreground`. Trocar um tom ou a ordem sem rodar o validador de novo é desfazer a medição.

6.3.3 3.3 A cor da marca: o dourado do monograma

`--primary` é o **dourado do R da logo**, medido do próprio arquivo: a letra é um gradiente, e o token é o meio da rampa dele — `oklch(0.78 0.085 80)`.

Nota de 24/08/2026 — a rampa deixou de ser amostrada e passou a ser lida. A medição original saiu do PNG por amostragem de pixel: `oklch(0.514 0.066 84)` no pé a `oklch(0.824 0.079 76)` no topo. Com a marca vetorial (`public/fraus-logo.svg`), a rampa vem das próprias paradas do gradiente e é exata: `oklch(0.642 0.084 76)` a `oklch(0.832 0.071 78)`. **O token não mudou** — `0.78 0.085 80` já caía dentro dela, e o argumento do croma fosco (0,085 contra os 0,15 do âmbar do "dito") sobrevive intacto. Registrado porque a fonte da verdade mudou de arquivo, não porque a cor mudou.

Por que deixou de ser lime. O `--primary` era `oklch(0.843 0.179 134)`, herdado do chassi. Nenhum pixel da logo é verde: a identidade é um monograma **F branco + R dourado sobre quase-preto**, e uma dashboard cuja cor de ação não existe na marca é uma identidade aplicada pela metade. O fundo grafite já concordava com o `#070707` do arquivo; faltava a cor de ação.

Por que isso não colide com o âmbar do "dito", que é semântico e intocável: o que separa os dois **não é o matiz, é o croma**. O dourado da marca é fosco (`0.085`) e o âmbar do dito é saturado (`0.15`) — metálico contra pigmento. Some-se a isso que eles nunca dividem superfície: `--primary` preenche **controle**, `--dito` marca **dado**.

A regra que sustenta essa convivência, e que não pode ser afrouxada: nenhuma série, categoria ou barra usa `--primary`. A única exceção nomeada é o **cursor de leitura** — o filete vertical que marca onde você está na linha do tempo, como no editor

de partitura —, e mesmo ele não codifica valor nenhum. Fora disso, `--primary` só aparece em ação primária e foco de teclado. Por isso **promotor é teal, não verde**: precisa estar longe do cursor.

`--warning (oklch(0.8 0.14 80))` compartilha o matiz do dourado. Ele sobrevive porque é **texto** (`--warning-rich-text`, o rótulo “suspeito” das métricas) e nunca preenchimento — e porque o croma o separa, pelo mesmo argumento acima. Se um dia surgir um botão de alerta preenchido, este é o par que precisa ser repensado primeiro.

6.3.4 3.3.1 O raio acompanha os cortes do monograma

`--radius` era `0.375rem`, e antes disso `0.625rem`. O F e o R são chanfrados a 45° e não têm uma curva de canto sequer; o raio de `0.625rem` arredondava o controle a ponto de ele não ter parentesco nenhum com a marca. Em `0.375rem` não ia a zero porque botão e campo totalmente quadrados brigariam com o chassi inteiro — cartão, popover, sheet — por causa de um detalhe.

Emenda de 24/08/2026 — o chassi endureceu, e a objeção caiu. `--radius` foi de `0.375rem` para **`0.125rem`** (2px), por decisão do dono do projeto. O argumento que segurava o raio em `0.375rem` era o chassi arredondado ao redor — cartão, popover, sheet, sidebar; era essa vizinhança que um botão quase quadrado brigaria com. Essa vizinhança não existe mais: os painéis grandes (`Painel.tsx`, `FaixaIndicadores.tsx`) passaram a ser chanfrados a 45° via `clip-path`, no mesmo desenho do F e do R, e a sombra migrou de `box-shadow` para `filter: drop-shadow()` porque `clip-path` descarta `box-shadow`. Com o chassi cortado a 45° em vez de arredondado, um raio quase reto deixou de ser a peça fora do lugar — passou a ser a única consistente.

6.3.5 3.3.2 O rótulo dentro do botão é verificado

`scripts/contraste.mjs` checava texto sobre fundo e **não checava texto sobre superfície preenchida** — ou seja, não checava o par mais clicado da interface. A lacuna apareceu justamente ao trocar o lime pelo dourado, que é o tipo de mudança capaz de derrubar a legibilidade do botão sem nenhum aviso. O script passou a cobrir `--primary-foreground` sobre `--primary` (hoje **9,64:1**) e o par equivalente da sidebar.

6.3.6 3.4 Categorias

Detrator, neutro e promotor mantêm cores distintas e **nunca** são comunicadas só por cor — sempre acompanham rótulo textual.

6.4 4. Tipografia

Uma família só, sans de sistema. O modo Operate tem permissão para isso, e o analista lê em DPI consistente: fonte de display em rótulo e dado é proibida.

Emenda de 31/08/2026 — a sans deixou de ser a de sistema. Etapa 1 do Pauta evoluído (docs/superpowers/specs/2026-08-31-pauta-evoluído-design.md), por decisão do dono do projeto: **Inter** variável na frente da pilha, com `font-feature-settings: "cv01", "ss03"` no body (o ajuste do Linear), e **JetBrains Mono** como `--fonte-mono` — o `.num` continua sendo quem liga numeral tabular por elemento. As duas entram por `next/font` no layout raiz: baixadas **no build** e servidas do próprio deploy, zero chamada de rede em runtime, que é a regra da casa. A pilha de sistema fica atrás como fallback. O que a regra original defendia sobrevive: continua sendo UMA família para rótulo e dado, e fonte de display segue proibida — a exceção nomeada é a **LP** (`app/page.tsx`), vitrine fora do modo Operate, onde o display usa **Mona Sans** variável em corpo grande com tracking até -0.02em . Ela entra por `next/font/local` a partir de `app/fontes/`, versionada no repo, e governa exatamente duas classes: `.display-vitrine` e `.titulo-vitrine`. `lib/tipografia.test.ts` varre o `globals.css` e barra qualquer TERCEIRA REGRA CSS que use `var(--fonte-display)` — essa via está coberta. Ele não pega classe utilitária arbitrária aplicada direto em JSX/TSX (algo como `className="font-[var(--fonte-display)]"`); cobrir esse caminho exigiria um harness de componente, que nenhuma dependência nova autoriza hoje. Essa via fica de vigilância manual em revisão de código, não de teste.

- **Escala fixa em rem**, razão apertada (1.125-1.2). Nada fluido.
- **Numeral tabular e monoespaçado em todo número que se compara** (`.num`). Número que dança ao atualizar é ruído, não dado.
- **Dinâmica por peso e tamanho**. Ênfase não gasta cor — a cor está reservada para dito/medido.
- Medida de prosa 65-75ch.

6.4.1 4.1 A prosa metodológica vira aparato

Os textos de honestidade **permanecem, todos**. O que muda é a posição: eles saem de cima do dado e vão para o **aparato** — o rodapé do sistema, na tipografia menor, onde a partitura põe nota de editor.

Regra: o dado aparece primeiro, a explicação fica a um gesto de distância e **nunca** empurra o dado para baixo da dobra. Recolher é permitido; remover não.

Os rótulos curtos — `estimativa`, `observado`, `sem sinal` — não são aparato: ficam colados ao número, sempre visíveis.

Emenda de 04/09/2026 — um aparato por tela, não um por painel. Com seis painéis numa tela, "Método e ressalvas" passou a se repetir seis vezes, idêntico, e repetição lê como ruído em vez de rigor. `AparatoDaTela.tsx` concentra as ressalvas de todos os painéis da página num único rodapé; cada painel continua produzindo a própria ressalva via `RessalvaDaTela`, só o **container visual** virou um. O que a regra original defendia continua de pé: **os rótulos curtos continuam fora do aparato**, colados ao número, sempre visíveis — só a prosa metodológica migrou para o container único.

A implementação usa portal, não estado copiado. A primeira versão deste arquivo guardava o JSX da ressalva em `useState` no momento do registro — e isso congelava a ressalva: um `FaixaIndicadores` que interpola `indicadores.comSinal` no rodapé ficava preso ao primeiro valor lido, mesmo com o número mudando na tela ao lado, o que é pior que a repetição que a mudança veio eliminar. `createPortal` resolve isso na raiz: o conteúdo nunca sai da árvore React do painel que o produz, só o destino no DOM é emprestado — ele volta a ser filho de verdade, e re-renderiza quando o painel de origem re-renderiza.

O aparato marca quando o sistema está em erro ou vazio. `RessalvaDaTela` aceita `estado?: "erro" | "vazio"` e imprime um rótulo curto (`não carregou` / `sem dado`) ao lado do título da ressalva no rodapé. Sem essa marcação, a nota metodológica leria como se tivesse se aplicado a um resultado que não existe — o texto continua verdadeiro (o sistema só não produziu dado desta vez), mas precisa dizer isso.

6.5 5. Estados

Todo componente interativo tem: padrão, hover, foco, ativo, desabilitado, carregando, erro e vazio. Não se entrega metade.

- **Carregando** é esqueleto com a forma do resultado, nunca roda girando.
- **Vazio** nomeia o que falta e qual etapa ou endpoint resolveria.
- **Falha isolada:** um sistema que não carrega mostra o próprio erro no lugar dele, e os vizinhos continuam de pé.
- **Sem sinal** é cabeça vazada, nunca zero, nunca cinza dentro da escala.

6.6 6. Movimento

150–250 ms, e **um momento autorado** em vez de efeitos espalhados: o cursor de leitura deslizando ao percorrer a linha do tempo. Movimento comunica estado, não decora. `prefers-reduced-motion` desliga o deslize e mantém o cursor estático.

Emenda de 24/08/2026 — a entrada dos sistemas. A regra original desta seção era "sem sequência orquestrada de entrada: o analista chega para trabalhar, não para assistir a página carregar". Ela foi afrouxada num ponto e mantida no resto, e a distinção é a seguinte:

- **continua proibido** animar na carga da página. Nada anima por ter montado;
- **passa a ser permitido** um sistema subir 8 px ao **entrar em cena pela rolagem**, uma única vez (`whileInView + once`). Isso não é abertura de cortina: é a mesma resposta que o olho já espera de algo que aparece por rolagem, e ela diz *de onde o bloco veio*.

A armadura de indicadores escalona os quatro filhos em 40 ms. O intervalo é curto de propósito: os 80–100 ms de praxe fariam o quarto indicador chegar quase meio segundo depois do primeiro, e aí o analista **espera** — que é exatamente o que a regra original proíbe. 40 ms lê como um gesto só.

O custo declarado: esta é a segunda peça de movimento da interface, e a seção dizia "um momento autorado". São dois agora — o cursor de leitura na linha do tempo e a entrada dos sistemas. Um terceiro precisa de argumento novo.

Nota de 25/08/2026 — a marca responde a gesto, e isso não é o terceiro momento. `MarcaFraus` reage a hover e a clique: o dourado varre pela fenda, os quatro primeiros cliques tremem, o quinto abre a marca. Parada, ela é **imóvel** — não anima por ter montado, não respira, não pulsa.

A distinção que sustenta isso: **momento autorado é o que a interface executa por conta própria** — o cursor de leitura, a entrada dos sistemas. Resposta a gesto do usuário é outra categoria, a mesma dos `transition-colors` de 150 ms que a navegação inteira já usa. Se hover de botão contasse como momento autorado, esta seção já estaria violada em cada linha da navegação. **A conta continua em dois.**

O vocabulário (duração, curva, distância) mora em `lib/movimento.ts`, um lugar só, pelo mesmo motivo que a cor mora em `token`. `prefers-reduced-motion` é respeitado em duas frentes, porque uma não alcança a outra: o bloco no fim do `globals.css` zera transição de CSS, e o `<Movimento>` no layout raiz passa `reducedMotion="user"` ao Motion, que anima em JS e não obedeceria ao CSS.

Emenda de 04/09/2026 — a coreografia de rolagem da vitrine, e por que ela não é o terceiro momento. A LP (`app/page.tsx`) ganhou uma cena que responde à posição de rolagem: `lib/cena.ts` devolve, por camada do ateliê (grade, planeta, campo de estrelas), um **fator** de 0 a 1 em função do progresso da rolagem, e `hooks/useProgressoDaCena.ts` escreve esse fator direto no nó raiz a cada quadro (sem `useState`, para não re-renderizar a árvore atrás de tabela e gráfico).

O argumento, e é decisão do dono do projeto: esta seção governa o modo **Operate** — a ferramenta que o analista lê por horas. A LP é vitrine declarada, visitada por segundos, com a atenção inteira na tela. O precedente já existe nesta própria seção, na nota de 25/08/2026 sobre a marca respondendo a gesto, e mais explicitamente na §8.7, que abriu a mesma exceção para o campo de partículas em WebGL: "a ferramenta é lida por horas e não pode ter GPU girando atrás do dado; a vitrine é visita de 40 segundos". A coreografia de rolagem é a mesma exceção aplicada a uma segunda peça da mesma vitrine — **a conta continua em dois na ferramenta:** o cursor de leitura e a entrada dos sistemas.

prefers-reduced-motion congela, não desacelera. Pela mesma decisão da §8.7, quem pede menos movimento não recebe uma versão lenta da cena — recebe a cena **parada no estado inicial** (`useProgressoDaCena` aplica o fator do topo da rolagem e não escuta o evento de scroll). Cena de rolagem devagar continua sendo cena de rolagem.

Correção da mesma emenda, 04/09/2026 — a armadilha do layout raiz. O parágrafo acima descrevia o desenho certo, mas o `Atelier` mora no layout **raiz** (`app/layout.tsx`), compartilhado por toda a árvore, e a primeira versão de `useProgressoDaCena` registrava o listener de `scroll` incondicionalmente. Resultado: a coreografia valia na ferramenta inteira, não só na vitrine — rolar `/dashboard/modelo` acendia e apagava grade, planeta e campo de estrelas atrás da tabela e do gráfico, que é exatamente a "decoração rodando o tempo todo" que esta seção proíbe, e o mesmo argumento da §8.7 que autoriza a exceção ("a vitrine é visita de 40 segundos") a condena fora dela. Estar no layout raiz não é a mesma coisa que estar só na LP; o código confundiu as duas coisas.

O conserto ficou na rota, num lugar só e nomeado. `lib/cena.ts` ganhou `ROTAS_COM_COREOGRAFIA` (hoje só `"/"`) e `coreografiaValeEm(pathname)`; `Atelier.tsx` lê `usePathname()` (ele já é client component) e passa o resultado para `useProgressoDaCena(ativo: boolean)`. Quando `ativo` é `false` — qualquer rota do Operate —, a função nem registra o listener de scroll: desligado é desligado, não "escutando e ignorando". Nos dois casos de desligamento (rota fora da vitrine, ou `prefers-reduced-motion`) o destino é o mesmo: `aplicar(0)`, o estado inicial em que os três fatores valem 1 — a cena de hoje, parada. O `Atelier` continua sendo uma peça só, a mesma instância nas duas bandas; só a reatividade é condicional.

6.7 7. A materialidade

Adotada em 21/08/2026. Ver `docs/superpowers/specs/2026-08-21-vidro-liquido-design.md`.

A superfície da interface é **vidro líquido**: quatro camadas com responsabilidades disjuntas.

Camada	O que é	Onde vive
0 — ateliê	manchas de luz fixas sobre o grafite	<code>components/shell/Atelier.tsx</code>
1 — vidro	superfície translúcida com quina iluminada	<code>.vidro-fino</code> , <code>.vidro</code> , <code>.vidro-denso</code>
2 — especular	realce que segue o ponteiro	<code>hooks/useEspecular.ts</code> + <code>.especular</code>
3 — dado	régua, séries, números — opaco, sem blur	os componentes de dado

A camada 0 não é decoração. Vidro sobre fundo chapado é indistinguível de cinza mais claro: sem luz atrás, não há o que refratar. Ela é estática, e essa é uma decisão — luz derivada do dado criaria um canal de cor sem rótulo, contra a seção 3.4 e contra a regra de que a cor da marca nunca codifica valor.

Três espessuras, não uma escala. `backdrop-filter` é o efeito mais caro de CSS, e cada espessura resolve um problema distinto: o fino sinaliza flutuação, o médio é superfície de trabalho, o denso existe porque num menu suspenso o conteúdo atrás **precisa** sumir para o rótulo da opção ser legível.

A regra dura: nada que carrega dado é translúcido. Gráfico, tabela, transcrição e régua desenham em superfície opaca embutida no vidro — um visor. Blur atrás de uma série de meio ponto come a série, e a régua da pauta é meio ponto.

O contraste governa o vidro, não o contrário. Cada espessura declara um `--vidro-<n>-piso`: a cor mais clara que ela pode assumir, com o vidro composto sobre a mancha mais brilhante do ateliê. É contra esse pior caso que `npm run contraste` verifica o texto. Par que reprova faz a espessura ganhar opacidade — o gate nunca é afrouxado. Se um piso passar de `--muted` (L 0.274) em claridade, o vidro clareou demais.

O especular tem dois desligamentos, e o segundo é o esquecido: `prefers-reduced-motion` e **ponteiro grosso**. Toque não tem hover, e o realce congelaria no último ponto tocado — pior que não existir.

6.8 8. Os temas

Emenda de 24/08/2026. Até aqui este documento dizia "tema escuro único, sem alternador", e a §3.1 chama o fundo de *papel de ensaio*, com o argumento explícito de que preto puro "transforma dado em neon". Passa a haver um segundo tema, **chuva de neon**, por decisão do dono do projeto — e ele é exatamente a estética que aquele parágrafo recusava.

Emenda de 01/09/2026, em dois tempos no mesmo dia. Primeiro entrou um terceiro tema, **espaço profundo**, pela mesma porta e pela mesma decisão. Ele não custou arquitetura nenhuma: `aplicarTema` já limpava a classe de todos os temas prevendo o caso, e os dois gates (`contraste.mjs` e `pisos.mjs`) descobrem os blocos `.tema-*` por regex — ele entrou na verificação sem uma linha de mudança nos scripts.

Depois o grafite foi DESCARTADO, também por decisão do dono do projeto, e o espaço tomou o lugar dele como padrão de fábrica. Padrão de fábrica mora no `:root` e é a AUSÊNCIA de classe (ver `lib/tema.ts`), então a promoção foi mover os 54 valores de chassi do bloco `.tema-espacial` para o `:root` e apagar o bloco. Quem tinha "grafite" no `localStorage` cai no padrão sozinho: `temaValido` só aceita o que existe, então a migração não precisou de código.

O que do grafite sobreviveu, porque a pergunta foi feita explicitamente: o **papel pautado** (`--pauta-*`), que era a tese dele e aqui vira linha de varredura de instrumento; e o **ouro fosco da marca** (`--primary` , croma 0,085, rampa medida do SVG), que não só sobreviveu como parou de precisar de justificativa — no espaço, a luz quente é literalmente uma estrela. Morreu só o chassi cinza-quente: fundo, superfície e borda.

Hoje são dois: **espaço profundo** (padrão) e **chuva de neon** (opção).

6.8.1 8.1 A tese do tema chuva

Rua molhada à noite: neon frio de fachada refletido no asfalto, e a luz de **sódio** do poste — que é âmbar-dourada. É isso que resolve o conflito entre vaporwave e a marca: numa cena de chuva noturna, luz quente de poste ao lado de neon frio não é contradição, é o retrato. O dourado do monograma deixa de ser um metal solto sobre roxo e passa a ser a lâmpada da rua.

6.8.2 8.2 O que o tema NÃO toca

Três regras, e elas eram o que separava "segundo tema" de "outro produto":

1. **A camada de dado inteira fica como está.** `--dito` continua âmbar, `--medido` continua azul, e as sete famílias de sinal mantêm os matizes. O magenta e o violeta do vaporwave vivem **só no ateliê**, que é a única camada que não carrega dado — e é por não carregar dado que ela pode ter croma alto sem inventar canal sem rótulo.
2. **O dourado continua fora de dado**, só em controle e marca.
3. **Todo par de texto continua cruzando AA por cálculo**, agora medido nos dois temas.

Emenda de 24/08/2026 — a regra 1 deixa de valer, e a atribuição dela estava errada. A regra 1 acima ("a camada de dado inteira fica como está") deixou de valer por decisão do dono do projeto: as sete famílias de sinal, as três categorias de NPS e os oito nós coloridos do grafo subiram croma e claridade.

O que continua de pé: o matiz de cada família ficou **congelado** — só saturação e claridade mudaram, e o matiz é o canal que carrega o encoding; saturação nunca carregou significado nenhum. `--no-importacao` continua cinza. `--primary` continua fora de dado, regra 2 intacta. Categoria continua nunca comunicada só por cor, sempre com rótulo textual — regra 3 intacta.

A correção de rota, e é a parte que importa: o texto acima chamava essa regra de "compromisso vinculante do `PRODUCT.md`". **Não é, e nunca foi.** Fomos reler o `PRODUCT.md`: ele exige duas coisas, e só duas — que todo par de cor que carrega texto cruze AA por cálculo, e que categoria nunca seja comunicada só por cor. Ambas continuam valendo e não foram violadas por esta emenda. "A camada de dado inteira fica como está" nunca esteve no `PRODUCT.md`; era decisão de design tomada pelo dono do projeto, e foi o mesmo dono do projeto que a revogou agora. Atribuir a um documento de produto uma regra que era decisão de design é o tipo de erro que este projeto não pode se dar ao luxo de deixar impresso.

6.8.3 8.3 O gate passou a ser ciente de tema

`scripts/contraste.mjs` mantinha **um mapa único** de tokens para o arquivo inteiro. Com dois temas declarando `--background`, o segundo sobrescreveria o primeiro e o relatório sairia verde tendo medido um tema só — falha silenciosa, que é o modo de falha que este gate existe para evitar.

Ele agora recorta o bloco de cada tema, **sobrepõe à base como a cascata do CSS faz** (um tema só declara o que muda) e roda a matriz inteira por tema. Tema novo entra sem tocar no script: basta o seletor começar com `.tema-`.

6.8.4 8.4 Onde a escolha mora

No `<html>`, como classe, e no `localStorage` — não em contexto de React. Quem precisa saber o tema é o CSS, e o CSS já sabe ler classe.

O grafite é a **ausência** de classe, não uma classe própria: ele é o `:root`, e um tema padrão que precisasse se declarar para funcionar quebraria em toda tela renderizada antes do JavaScript. Um script inline no `<head>` aplica a classe antes da primeira pintura; sem ele, quem escolheu a chuva veria a tela pintar em grafite e trocar depois da hidratação, a cada navegação.

6.8.5 8.5 Onde a chuva pode gritar

O tema foi empurrado para ser chamativo em 24/08/2026, e o empurrão respeita a regra de 8.2: **só o que não carrega dado ganhou saturação**.

Nota de 24/08/2026 — a tabela vale para os dois temas. O título desta seção pressupõe um tema só, e não é bem assim: a decisão de 24/08/2026 foi **estrutura igual nos dois temas** — cada peça abaixo existe no componente tanto no grafite quanto na chuva, e é o tema, não o componente, quem decide cor e intensidade. No grafite **só asfalto e chuva ficam em zero**; as outras seis estão acesas, apenas contidas — marca impressa em 0,15, grade a laser em 0,1, sol listrado em 0,16, papel pautado em 0,12, e quina e sombra ativas com suas próprias cores. É a chuva que grita, não o grafite que apaga.

Peça	O que faz	Por que pode gritar
quina do vidro	ciano na aresta de cima, magenta na de baixo	é tubo de neon: a luz da fachada bate em cima, o reflexo do chão sobe embaixo. Toda superfície tem quina, então acender a aresta acende a interface inteira sem uma célula de dado mudar de cor
sombra	deixa de ser preta e vira derrame violeta	preto sob um painel apoiado em asfalto molhado lê como buraco; o halo lê como a luz do próprio painel vazando para o chão
especular	tingido de magenta	realce branco no meio de uma paleta tingida lê como falha de renderização
reflexo no asfalto	faixa de luz subindo do rodapé	é o que separa "fundo roxo com manchas" de "fachada espelhada em chão molhado"
chuva	riscos diagonais finos, estáticos	dá textura para o vidro refratar sem gastar quadro
marca impressa	o monograma sangrando pela quina inferior direita, em magenta	é máscara pintada pelo tema, não imagem: mora no ateliê, não codifica valor nenhum, e o magenta é o mesmo par de néon da quina do vidro
grade a laser	grade em perspectiva convergindo para um horizonte	é o piso do vaporwave clássico: converte o ateliê em cenário de fuga, sem tocar dado
sol listrado	sol com faixas horizontais no horizonte	a segunda peça do mesmo cenário; nos dois temas, com intensidade decidida por tema

A chuva não cai, e isso é decisão: a §6 já gastou os dois momentos de movimento que a interface se permite, e chuva animada seria um terceiro que não comunica estado nenhum — decoração rodando atrás de tabela e gráfico o tempo todo.

As duas peças novas existem no componente nos **dois** temas; o grafite as mantém em opacidade zero. Um segundo componente de ateliê só para a variante custaria mais do que dois tokens.

6.8.6 8.6 O preço: o vidro da chuva é mais grosso

Nota de 04/09/2026 — os pisos "no grafite" citados nesta seção são históricos. O grafite é o tema descartado pela §8 em 01/09/2026; nenhum valor de piso, espessura de vidro ou contraste medido "no grafite" existe em produção. Ficam registrados pelo método que documentam — a conta que a §7 exige refazer sempre que o ateliê ganha luz nova —, não como referência de um tema vigente. Os pisos vigentes são os do **espaço profundo** (§8.7) e da **chuva de neon**, remediados na emenda de 04/09/2026 ao fim desta seção.

Acender o ateliê levantou o piso das superfícies translúcidas, e o portão reprovou três pares — ~~---destructive~~ sobre vidro fino caiu para **3,53:1**.

A resolução seguiu a regra da seção 7: **a espessura ganha opacidade até passar**. 45% → 68% no fino, 62% → 74% no médio, 82% → 86% no denso. Baixar a luz para salvar o contraste teria sido resolver pelo lado errado, ainda mais num tema cujo pedido era justamente ficar mais chamativo.

Os pisos deste tema são **medidos, não estimados**: as camadas do ateliê compostas no mesmo ponto (pior caso, ainda que geometricamente impossível — a mancha da marca fica no alto à esquerda e a fria embaixo à direita), cada espessura por cima, e

a luminância resultante convertida de volta para OKLCH. Piso otimista é pior que piso nenhum: ele faz o portão devolver verde medindo uma superfície que não existe.

Emenda de 24/08/2026 — a medição virou script, e ela estava errada.

Esta conta já tinha sido feita três vezes à mão, e nas duas primeiras morreu num scratchpad. Agora é `dashboard/scripts/pisos.mjs` (`npm run pisos`): ele lê o `globals.css`, compõe o ateliê, aplica cada espessura e imprime o piso que cada uma deveria ter. Medição sem instrumento versionado vira chute na terceira rodada — e virou.

Refeita a conta com o script, os pisos da chuva registrados aqui (0,307/0,294/0,257) eram **otimistas mesmo para o ateliê daquela época**: o vidro fino dava 0,333. O portão vinha medindo uma superfície mais escura que a real, ou seja, devolvendo verde com folga que não existia. É exatamente o modo de falha que o parágrafo acima descreve, e ele aconteceu aqui. Os valores vigentes estão no `globals.css` e batem com o script.

Um pessimismo a menos, e ele é lido da geometria. O papel pautado **não soma** com as peças do rodapé: a máscara dele (`radial-gradient(140% 100% at 50% -10%, black 15%, transparent 70%)`) zera por volta de 60% da altura da tela, então onde ele tem força as outras ainda não começaram. O script disputa "papel pautado" contra "marca impressa + grade a laser" e fica com a composição mais clara. O resto do pior caso segue impossível de propósito (as manchas de luz no mesmo ponto), porque ali a impossibilidade é barata.

Correção da revisão final (24/08/2026) — a marca e a grade NÃO eram disjuntas, e esta emenda afirmava que eram. A versão anterior deste parágrafo dava como prova a máscara antiga da marca (`radial-gradient(120% 120% at 100% 100%, ..., transparent 72%)`), estreita o bastante para deixar massa só num raio pequeno. Essa máscara foi **alargada depois**, de propósito, para a marca ficar visível: hoje é `radial-gradient(150% 150% at 100% 100%, black 34%, transparent 96%)` num elemento de 115vmin ancorado na quina inferior direita — e a grade ocupa a faixa `bottom-0 h-[45vh]` de largura total, mais forte justamente colada no rodapé. O pixel "linha da grade sobre traço do monograma" existe. O modelo de medição não acompanhou a mudança de geometria, e por isso os seis pisos ficaram otimistas. Corrigido: **marca e grade somam**; só o papel pautado continua disjunto. Premissa de disjunção é afirmação sobre geometria — quem mexer numa máscara do `Atelier.tsx` reconfe a lista em `pisos.mjs`.

O que continua sendo julgamento de olho. O ateliê só aparece nas calhas entre painéis e **através** do vidro; numa tela cheia, engrossar o vidro apaga a luz que a emenda acabou de acender. A intensidade da marca foi posta no teto em que ela ainda **não** é a camada que manda no piso — 0,15 no grafite, 0,22 na chuva —, e a presença dela veio de **escala** (115vmin), que é de graça: o pior caso já assume a cor da marca em algum pixel, então o tamanho não custa contraste nenhum.

Emenda de 24/08/2026 (segunda) — os números finais, e um par sem folga nenhuma. A grade, o sol e a marca impressa (0,12/0,22) levantaram o piso de novo, e a resolução foi a mesma de sempre: a espessura cede. Vidro fino do grafite foi de 55% para **64%**; médio (72%) e denso (82%) do grafite ficaram como estavam. Na chuva, vidro fino foi de 71% para **74%** e vidro médio **recuou** de 75% para **74%** — ele chegou a subir para 81%/85% no meio do caminho e voltou quando um bug de medição foi corrigido; denso (86%) não mudou. Contra o início desta sessão, o saldo da chuva é: fino subiu 3 pontos, médio caiu 1.

Um par ficou exatamente no fio. `--destructive` sobre o piso do vidro fino do grafite fecha em **4,50:1** — o limiar do WCAG AA, não um número acima dele. Zero folga: qualquer luz nova no ateliê do grafite — mais uma peça, mais opacidade em alguma das que já existem — derruba esse par sem precisar de erro nenhum no cálculo. Fica registrado como o candidato mais provável a reprovar na próxima rodada.

As nove cores que carregam texto foram convertidas de `hsl()` para `oklch()`. É conversão matematicamente equivalente, não repintura — o motivo foi o portão: `scripts/contraste.mjs` precisava enxergá-las para medir, e três delas nunca tinham sido medidas em tema nenhum antes desta sessão. As nove passam AA nos dois temas, incluindo essas três.

Emenda de 24/08/2026 (terceira) — os números depois de consertar a disjunção. Com marca e grade somadas (ver a correção acima), os seis pisos ficaram otimistas e o portão reprovou: `--destructive` × vidro fino caiu para **4,29** no grafite e **4,39** no fino e no médio da chuva. A regra de sempre: a espessura cede, a luz não. Vidro fino do grafite subiu de 1 em 1 — 64% → 65% → 66% → 67% → 68% → **69%** —, remedindo o piso a cada passo; médio (72%) e denso (82%) do grafite não mudaram. Na chuva, fino e médio subiram juntos de 74% para **76%**; denso (86%) não mudou.

O par crítico deixou o fio: `--destructive` × piso do vidro fino fecha em **4,54** no grafite e **4,52** na chuva. Continua sendo o par mais apertado da interface e o primeiro candidato a reprovar se o ateliê ganhar luz nova.

Emenda de 24/08/2026 (quarta) — o vidro afina, e o vermelho de erro paga a conta. Por decisão do dono do projeto: *"vamos diminuir a opacidade dos liquid glass para que eu consiga ver o fundo"*.

A folga era zero. Medido antes de mexer: dois pontos a menos em qualquer espessura já levavam `--destructive` × vidro fino de 4,52 para 4,44. O par que a emenda anterior deixou "no fio" era literalmente o teto.

E o teto não é o vidro, é o texto — esta é a parte que vale registrar, porque ela contraria a intuição. O que se enxerga do ateliê *através* de uma superfície translúcida **é** a claridade do piso dela. Pedir "mais fundo visível" é pedir piso mais claro, e piso mais claro exige texto mais claro. Não adianta afinar o vidro e escurecer o ateliê para compensar: as duas coisas são a mesma grandeza, e o portão mede exatamente ela. **A única alavanca que compra fundo é subir a cor do texto que fica por cima.**

Isso **não afrouxa o portão** — conserta a cor do texto, que é o outro lado da mesma regra da §7. O portão continua em 4,5 e mede tudo que media antes.

`--destructive` foi de `oklch(0.704 0.191 22.216)` para `oklch(0.79 0.121 22.216)`. **O matiz não se moveu:** vermelho continua sendo erro. É seguro porque `bg-destructive` cheio só pinta bolinha de status — o botão e o badge usam `/10` e `/20` com `text-destructive` por cima, então clarear melhora os dois usos e não há rótulo sobre preenchimento sólido.

O croma caiu de propósito. O valor antigo estava **fora do gamut sRGB** ($R = 1,013$) e o navegador já o cortava; subir a claridade mantendo 0,191 pioraria isso ($R = 1,319$). 0,121 é o croma máximo dentro do gamut nessa claridade — `rgb(255,83,79)` —, então o token passa a declarar a cor que de fato renderiza, e ainda rende um pouco mais de contraste que a versão cortada. Token que declara cor que o navegador corta é a mesma classe de defeito de documento que afirma o que o código não faz.

O que isso comprou: oito pontos de vidro nas duas espessuras que o usuário enxerga. Grafite fino **69% → 61%**, médio **72% → 64%**. Chuva fino e médio **76% → 68%**. Denso não mudou em tema nenhum — ele existe para o conteúdo atrás sumir, e afiná-lo tem custo funcional, não só de contraste.

O gargalo trocou de dono, e o novo tem outro nome. O par mais apertado agora é `--detrator-texto` × piso do vidro fino, em **4,54** na chuva e **4,72** no grafite. `--detrator-texto` é **cor de dado** — a categoria de NPS —, não cromo. Afinar mais exigiria mexer na camada de dado, e isso é decisão de outra ordem: fica registrado aqui como a fronteira, não como pendência.

Emenda de 04/09/2026 — o pior caso virou trajetória, e nenhum piso mudou. Até aqui esta seção media o ateliê num único ponto (pior caso geométrico fixo). A §6 desta sessão deu à LP uma cena que responde à rolagem (`lib/cena.ts`), e isso trocou o pior caso de um ponto por uma **trajetória**: a claridade do ateliê varia ao longo da rolagem, e o piso de vidro precisa valer para o ponto mais claro dessa trajetória inteira, não só do herói.

`scripts/pisos.mjs` passou a ler `FATOR_MAXIMO_POR_CAMADA` de `lib/cena.ts` (por expressão regular — o próprio script documenta em maiúsculas que importar de verdade exigiria transpilar, e é justamente por isso que o script recorta o bloco do arquivo-fonte em vez de fazer `import`) e multiplicar cada fator pelo token de opacidade do tema vigente (`--grade-op`, `--sol-op`, `--estrelas-op`), em vez de ler só a opacidade estática do CSS — o mesmo motivo que fez `intensidadeDaCamada` nascer função pura em `lib/cena.ts`: fator escondido em JSX é fator que o script não lê, e o portão volta a medir uma superfície que não existe, como já aconteceu duas vezes nesta mesma seção.

E nenhum piso mudou. O motivo está na própria função: `cena.ts` devolve **fator** (0 a 1), não opacidade, e os três fatores valem exatamente 1 no topo da página (`progresso = 0`) — decisão registrada no código como correção de 04/09/2026, para o herói ficar idêntico ao de hoje pixel por pixel. Como o herói já era o ponto usado para medir o pior caso, e nenhum fator ultrapassa 1 em ponto nenhum da rolagem, a cena **nunca fica mais clara que o herói de hoje** — a trajetória inteira está contida no ponto que já era medido. O risco que a Task 12 antecipava — `--detrator-texto` × vidro fino em 4,54 na chuva, o par mais apertado da interface, cedendo com a nova fonte de luz — não se materializou.

`npm run contraste`: **348 PASS / 0 FAIL** nos dois temas vigentes (espaço profundo e chuva de neon). O par mais apertado continua sendo `--detrator-texto` × `--vidro-fino-piso` / `--vidro-piso`, em **4,54** na chuva — o mesmo valor, sem folga nova nem perdida.

6.8.7 8.7 O padrão de fábrica: espaço profundo (01/09/2026)

A tese: o vazio visto de dentro de uma cabine. Não é "fundo preto com estrelinhas". A cena tem três corpos, e cada peça do ateliê já existente virou um deles — **nenhuma peça nova foi criada** exceto o campo de estrelas:

corpo	peça	por quê
a estrela	mancha da marca (--ateliê-marca)	no espaço a luz quente é literalmente uma estrela. A chuva precisou argumentar que o ouro era a lâmpada de sódio; aqui o ouro é o corpo luminoso, sem empréstimo
a nebulosa	mancha fria (--ateliê-fria)	croma alto pode, porque esta camada não carrega dado
o planeta	sol listrado (--sol-*)	o disco com faixas lê como gigante gasoso assim que a paleta esfria. Mesma geometria, outra leitura

Como ele não vira "chuva 2" — o risco real de um terceiro tema escuro e roxo: a chuva é urbana, saturada e **cheia**; este é vasto, escuro e **vazio**. O croma do chassi cai, a densidade de luz cai (--ateliê-op-* são os menores dos três) e o que ocupa a tela é o espaço negativo entre poucos corpos. São opostos de temperamento, não vizinhos de matiz.

As **três regras da §8.2 valem sem emenda**: camada de dado intacta, dourado fora de dado, todo par de texto cruzando AA por cálculo — nos dois temas vigentes.

O vidro deste tema é o mais FINO dos três (56%, contra 61/64/82 no grafite e 68/68/86 na chuva), e isso inverte a §8.6 sem contradizê-la. A regra continua a mesma — a espessura responde ao piso, o portão manda no vidro. O que mudou foi o piso: ateliê menos aceso significa composto mais escuro, e composto mais escuro deixa o vidro afinar sem ameaçar texto nenhum. O número saiu do exercício, descendo de 4 em 4 e remedindo a cada passo: 68% → 64% → 60% → **56%** (último que passa) → 52% (reprova).

Duas correções pagas na tela, e não no token — as duas invisíveis para os gates, que é exatamente o que a §9 avisa:

- O planeta chegou cinza.** As faixas do disco são `oklch(0 0 0 / 0.85)` cravadas no `Atelier.tsx` — barras pretas quase opacas. Sobre um azul-gelo de croma 0,065 elas produzem uma média dessaturada, e o disco lia como borrão listrado atrás da manchete. Corrigido subindo o croma (a cor que passa por peneira preta tem que entrar mais forte do que se quer ver), engrossando a faixa de 9px para 15px (nove pixels nesta escala viram moiré, banda larga lê como gigante gasoso) e baixando a opacidade de 0,26 para 0,15, porque cenário não disputa leitura com o único texto que a página precisa entregar.
- O campo de partículas da LP nasceu tímido demais** para ser percebido, e subiu de 170/62 para 260/105 em contagem e tamanho de base.

O campo de estrelas é a única peça nova, e ela é CSS estático: três tiles de períodos primos entre si (137, 191, 89 px) com brilhos diferentes. Períodos sem divisor comum só repetem o conjunto a cada 137×191×89 px — ou seja, nunca dentro de uma tela —, e um `background-repeat` único ladrilharia visivelmente. O brilho variado é o que dá profundidade: estrela toda do mesmo peso lê como ruído de sensor.

Ele é **estático** pela §6, como a chuva e a grade. O espetáculo em WebGL existe — `components/lp/CampoDeParticulas.tsx` —, mas **só na LP**, e a distinção é de contrato, não de gosto: a ferramenta é lida por horas e não pode ter GPU girando atrás do dado; a vitrine é visita de 40 segundos com a atenção inteira na tela, e ali o espetáculo é o produto. Quem pede `prefers-reduced-motion: reduce` não recebe uma versão mais lenta — recebe `null`, porque campo de partículas lento continua sendo campo de partículas.

Nota para `pisos.mjs`. O campo de estrelas entrou na lista de camadas com a contribuição **ponderada pela cobertura areal** ($\sim 8 \times 10^{-5}$), e não pela opacidade do token. As outras camadas são lavagens de tela cheia, onde a opacidade já é a contribuição; esta são três pontos de $\sim 1,5\text{px}^2$ em tiles de $\sim 18.800\text{px}^2$. A média é o número certo porque o que o script modela é o fundo visto através de `backdrop-filter: blur()`, e blur devolve a média local — medir a estrela pelo pico seria medir um pixel que o vidro nunca entrega.

6.9 9. O ciclo de verificação, e o que cada gate NÃO vê

Cinco comandos verificam este sistema de design. Eles não se substituem, e a seção existe porque **quatro deles já devolveram verde com o build quebrado**.

Comando	O que ele vê	O que ele não vê
<code>npm run pisos</code>	se o piso gravado no CSS bate com o pior caso medido do ateliê	qualquer coisa fora dos tokens que ele casa por regex
<code>npm run contraste</code>	todo par de cor que carrega texto ou marca, em todos os temas	cor escrita em notação que o parser não lê; matiz; a tela
<code>npx tsc --noEmit</code>	tipos do TypeScript	CSS, imagem, qualquer asset
<code>npm run lint</code>	as regras do ESLint	CSS, imagem, qualquer asset
<code>npm run build</code>	compila o CSS e processa os assets de verdade	a tela

`npm run build` **faz parte do ciclo, não só da conferência final**. Em 24/08/2026 ele pegou duas quebras no mesmo dia que os outros quatro deixaram passar em verde:

- um `favicon.ico` gerado a partir de imagem **RGB** — o processador do Next exige RGBA e falha com `The PNG is not in RGBA format!`;
- um `*/` que fechou um comentário do `globals.css` cedo demais, deixando o resto do texto como CSS solto (`Unknown word lime`).

Nos dois casos `pisos`, `contraste`, `tsc` e `lint` passaram limpos. Os dois primeiros só casam token por expressão regular; os dois últimos não leem CSS nem imagem. **Nenhum dos quatro compila coisa alguma**.

E nenhum dos cinco vê a tela. Uma peça de fundo pode estar geometricamente quebrada, invisível por opacidade baixa demais, ou ilegível por figura contra fundo de mesmo matiz, com os cinco verdes. Foi assim que o papel pautado saiu invisível a 5,5%, que a grade a laser colapsou numa faixa de 30px, e que o portão passou a medir uma superfície que não existe. **Trabalho visual se confere olhando**, nos dois temas, numa tela cheia e numa vazia.

7. Conformidade regulatória — o Fraus e o EU AI Act

Escrito em 10/09/2026. Este documento existe porque o Fraus **classifica emoção**, e isso tem nome no regulamento europeu. Ele descreve o escopo, o que o sistema infere, o que ele nunca decide sozinho e — a parte que virou regra de código — **o que ele está proibido de fazer**.

7.1 1. Por que este documento existe

`fraus/sinais/emocao.py` classifica **sete emoções** (as de Ekman) e deriva **desprezo** da díade raiva + nojo. Pela letra do Regulamento (UE) 2024/1689 — o *AI Act* — isso é um **sistema de reconhecimento de emoções**: um sistema de IA destinado a identificar ou inferir emoções de pessoas naturais a partir de dados biométricos ou comportamentais.

Não importa que o Fraus rode local, sem LLM e sem câmera. O que classifica o sistema é **o que ele infere sobre pessoas**, não o tamanho do modelo.

7.2 2. Escopo do sistema

O que analisa	transcrições de atendimento por chatbot, já encerradas
Sobre quem infere	o cliente — e apenas o cliente
O que infere	satisfação (score 0-100 → nota 0-10 → categoria NPS), emoção por mensagem, ironia por mensagem, indicadores agregados
Quando roda	em lote, sobre conversa terminada. Nunca em tempo real durante o atendimento
O que decide sozinho	nada
Base do modelo	três BERTimbau fine-tunados + regressão logística, inferência local em CPU

7.3 3. As três obrigações que já valem, e a que vem

7.3.1 3.1 Transparência — Art. 50(3), desde 02/08/2026

O regulamento exige que o *deployer* de um sistema de reconhecimento de emoções **informe as pessoas naturais expostas** a ele.

Como o Fraus cumpre: toda tela onde a leitura de emoção aparece (`components/CabecasDeLeitura.tsx` , compartilhado pelo simulador e pela análise) carrega a etiqueta de exposição, em linguagem de gente, junto da nota metodológica que já qualificava o NPS como inferido. A obrigação de *informar o cliente final* é de quem opera o chatbot — o Fraus fornece o texto e diz que ela existe; ele não fala com o cliente.

7.3.2 3.2 Proibição no local de trabalho — desde 02/02/2025

Reconhecimento de emoção **no ambiente de trabalho é proibido**, sem exceção comercial. Isso não é obrigação a cumprir: é uma fronteira que o produto não pode cruzar.

7.3.3 Regra: o Fraus não pontua atendentes.

O atendente humano aparece no transcript — a escalação é parte do dado — e o classificador de texto **recusa pontuar mensagem que não seja do cliente** (`sinais/texto.py` ; `qtd_humano` e `latencia_mediana_humano_s` são contagem e relógio, nunca julgamento). "Score de performance por atendente" é a sugestão mais recorrente em analytics de atendimento, e no Fraus ela está **fechada por desenho**, não esquecida no backlog.

Vale com o mesmo peso das invariantes do `CLAUDE.md`: implementar isso não é feature nova, é violação.

7.3.4 3.3 Alto risco (Anexo III) — aplicação em 02/12/2027

Reconhecimento de emoção voltado ao **cliente** é permitido, mas reclassificado como alto risco, com obrigações que entram em 02/12/2027: avaliação de conformidade, supervisão humana, logging, avaliação de impacto em direitos fundamentais e monitoramento pós-mercado.

Onde o Fraus já está, hoje, em relação a cada uma:

Obrigação	Estado no Fraus
Supervisão humana	o sistema não decide nada. O analista curou o léxico (<code>sinais/curadoria.py</code>), move as faixas de NPS (<code>PUT /configuracoes</code>) e lê os transcritos. Nenhuma saída aciona ação automática sobre ninguém
Logging	<code>entregas_webhook</code> registra toda entrega e toda recusa, com veredito e motivo. É o embrião da trilha de auditoria
Transparência	etiqueta de estimativa em todo número inferido; limitações declaradas em tela e em <code>docs/treinamento.md</code>
Monitoramento pós-mercado	<code>GET /saude/deriva</code> reporta quais features estão fora da faixa de distribuição em que o modelo foi treinado
Avaliação de conformidade	não feita. É trabalho de quem for colocar isto em produção comercial, e está fora do escopo do TCC
Avaliação de impacto em direitos fundamentais	não feita, pelo mesmo motivo

Isto não é uma declaração de conformidade. É um mapeamento honesto do que já está de pé e do que não está.

7.4 4. Limitações que importam para quem lê um número do Fraus

- **O NPS é inferido do texto, não perguntado.** O intervalo de confiança que a interface mostra cobre a **incerteza amostral** — não a incerteza do modelo.
- **A cabeça de ironia é pouco confiável** e por isso **não pontua** desde 04/09/2026: no corpus de treino ela funciona como detector de sentimento positivo. Continua sendo exibida, com a ressalva colada.
- **O sinal de tempo é treinado em dados sintéticos**, calibrados por literatura, porque nenhum corpus público em português tem timestamps de diálogo. Fora da faixa de latência do treino, o relógio domina a nota — é limitação medida e declarada, e `GET /saude/deriva` a torna visível.

- **Desprezo não é classe treinada**; é derivado da díade raiva + nojo.
- **Ausência de dado nunca vira zero**. Conversa sem fala do cliente sai "sem sinal", em toda superfície.

7.5 5. Fontes

- [Regulamento \(UE\) 2024/1689 — AI Act](#)
- [Art. 50 — obrigações de transparência](#)
- [Diretrizes de transparência da Comissão Europeia](#)
- [Obrigações de transparência em vigor desde 02/08/2026 — Cooley](#)
- [Emotion AI de cliente vira alto risco — CX Today](#)

8. Encolher os modelos — o que foi medido, e por que a int8 foi recusada

Medido em 10/09/2026, na máquina de desenvolvimento, contra os checkpoints reais. Nada aqui exigiu retreino: quantização dinâmica é transformação **pós-treino** sobre os pesos já aprendidos.

8.1 O problema

3 BERTimbau (fp32)	3 × 416 MB = 1.249 MB
<code>torch</code> que os executa	497 MB
imagem de inferência	~1,75 GB

Isso limita hospedagem, encarece o container e — desde que o teto de função Python da Vercel subiu para 500 MB (24/02/2026) — é a única coisa entre o Fraus e um deploy serverless.

8.2 O que foi testado

Quantização dinâmica int8, sobre `bertimbau-satisfacao`, com as mesmas dez sondas em todas as variantes:

estratégia	tamanho	max Δp	classe mudou
qint8 em tudo	105,5 MB	0,4535	1 sonda
qint8 só em MatMul	172,0 MB	0,4953	1 sonda
qint8 MatMul + <code>per_channel</code>	172,4 MB	0,1385	nenhuma
qint8 só em MatMul	172,0 MB	0,2620	1 sonda
qint8 em tudo + <code>per_channel</code>	105,9 MB	0,1015	nenhuma
qint8 tudo + <code>per_channel</code> + <code>reduce_range</code>	105,9 MB	0,1798	nenhuma

Duas coisas valem ser registradas porque contrariam a intuição comum:

- `per_channel` **é o que decide**, não a escolha de quais operadores quantizar. Ele dá a cada coluna de peso a própria faixa, em vez de espremer a matriz inteira numa só — e numa camada de atenção as colunas têm magnitudes muito diferentes.
- "não quantize o embedding, só MatMul" saiu pior nas duas pontas**: 63% maior e menos fiel. Foi medido, não herdado de tutorial.

8.3 O resultado da melhor variante

Aplicada aos três modelos:

modelo	antes	depois	redução	max Δp	classe mudou
satisfacao	416.2M	105.9M	3.93x	0.1015	nenhuma
emocao	416.2M	105.9M	3.93x	0.1129	nenhuma
ironia	416.2M	105.8M	3.93x	0.1486	nenhuma
TOTAL	1248.6M	317.6M	3.93x		

Velocidade: **2,10x mais rápido** em CPU (309 ms → 147 ms por lote de 10 frases, `bertimbau-satisfacao`).

Pela medida de classificador — nenhuma sonda mudou de classe vencedora — isso passaria.

8.4 E é aqui que ela é recusada

Diferença de probabilidade é a medida errada para este sistema. O score do Fraus é uma *projeção* das três probabilidades num eixo 0-100; a nota é o score arredondado; a categoria é a nota caindo numa faixa. Cada degrau absorve ou **amplifica** o erro do anterior.

Medido ponta a ponta, com o motor inteiro montado dos dois jeitos, sobre 180 conversas determinísticas do simulador:

desvio médio de score	0,83 pontos
desvio máximo	25,65 pontos
notas diferentes	16 de 180 (8,9%)
categorias diferentes	3 de 180 (1,7%)

E o caso que encerra a discussão:

```
sim-0-635276501 59,95 -> 85,61 detrator -> promotor
```

Uma conversa atravessa **duas faixas** e troca de veredito. Categoria é o que a tela mostra e o que o NPS agrega: trocar o backend não pode trocar o veredito.

Decisão: os artefatos int8 NÃO são promovidos. Eles ficam em `modelos-onnx/` (fora do git) para quem quiser reproduzir a medição, e o portão de `scripts/comparar_backends.py` continua vermelho de propósito.

8.5 A variante mista, medida em 11/09/2026

A tabela de caminhos abaixo listava "fp32 na satisfação, int8 no resto" como **estimativa**, nunca medida. A hipótese era que o flip vinha de quantizar o modelo que *governa* o score, e que poupá-lo resolveria. Foi medida, com o mesmo portão e as mesmas 180 conversas:

	int8 nos três	mista
desvio médio de score	0,83	0,356
desvio máximo	25,65	18,99
notas diferentes	16 de 180	6 de 180
categorias diferentes	3 de 180	2 de 180

A hipótese estava metade certa, e metade não basta. Poupar a satisfação corta o dano quase pela metade — e ainda troca o veredito de duas conversas. A mesma `sim-0-635276501` do caso original segue atravessando faixa (59,95 → 78,94).

8.5.1 De quem é a culpa, então

A ironia **saiu do vetor do Fusor** em 04/09/2026 (ver `NOMES_FEATURES`), então quantizá-la não pode mover score nenhum. Isso deixa apenas a emoção como suspeita, e a medição confirma — satisfação fp32 + emoção fp32 + **ironia int8**:

```
desvio medio de score : 0.000 pontos
desvio MAXIMO        : 0.000 pontos
notas diferentes      : 0 de 180
CATEGORIAS diferentes : 0 de 180
ACEITO
```

Zero exato, nas 180. Duas coisas ficam provadas de uma vez:

1. **ONNX fp32 é troca de executor, não de modelo.** Δp máximo 0,0000 por modelo, desvio 0,000 ponta a ponta. O que a conversão custa é nada.
2. **A única cabeça quantizável de graça é a ironia**, e exatamente porque ela não pontua. As duas que pontuam não toleram int8.

8.5.2 E o que isso faz com a máquina de 1 GB

Nada de bom. O `VM.Standard.E2.1.Micro` (o único shape com capacidade em sa-saopaulo-1 em 11/09/2026) tem 1 GB de RAM, e o sistema operacional come ~200:

variante	pesos	veredito	cabe em ~800 MB
ONNX fp32 nos três	1.250 MB	✓ idêntico	✗
fp32 + fp32 + ironia int8	939 MB	✓ desvio 0,000	✗
fp32 satisfação + int8 nas outras	628 MB	✗ 2/180 trocam	⚠ no limiar
int8 nos três	318 MB	✗ 3/180 trocam	✓

A ordem é cruelmente monótona: **tudo que cabe troca veredito, e tudo que preserva o veredito não cabe**. Não há ajuste no meio — a emoção precisa ficar em fp32, e aí são dois codificadores de 416 MB, que já estouram o orçamento sozinhos.

⚠ Medição de RAM pendente, e por que o número preliminar não serve

Medindo o processo com o motor montado, o ONNX consumiu **mais** residente que o torch (pico 1.366 MB contra 1.075 MB): o `onnxruntime` carrega o grafo inteiro de imediato, enquanto o torch mapeia os `safetensors` e só paga as páginas que toca.

Esse número **não está limpo**: a sonda importava `fraus.motor` e `fraus.fusor` antes de escolher o backend, e são eles que arrastam torch e sklearn — 354 MB de base nos dois casos. Medir o caminho ONNX de verdade exige uma `venv` **sem** torch, e hoje `torch>=2.3` é dependência **principal** no `pyproject.toml`: a imagem sem torch que o extra `onnx` promete não existe sem mover essa dependência para um extra.

A conclusão acima não depende disso — 939 MB de peso já passam de 800 —, mas a promessa de "ONNX corta a imagem em 2,6x" continua **não verificada em RAM**, só em disco.

8.5.3 O que sobra de ganho real

Independente da máquina pequena, a variante aceita **é um ganho para o deploy na A1**: desvio 0,000, e a imagem perde os 497 MB do `torch` em disco (depois de mover a dependência). Na sessão de 11/09/2026 o envio do código e dos pesos para a VM era de 1,3 GB — cortar isso quase pela metade é menos tempo de deploy e menos coisa para dar errado no meio.

E se 1 GB algum dia virar requisito de verdade, o caminho não é quantização: **é um codificador com três cabeças**. Os três checkpoints são ajustes finos do mesmo BERTimbau, então os 110M de parâmetros estão duplicados três vezes. Um encoder compartilhado com três cabeças de classificação daria ~416 MB em precisão cheia — cabe em 1 GB sem quantizar nada e sem risco de veredito. O preço é retreino, e portanto a invariante 10 de novo; não é ajuste de deploy.

8.6 O backend ONNX em produção, medido em 15/09/2026

A variante aceita acima (satisfação e emoção fp32, ironia int8) virou um executor de verdade: `FRAUS_BACKEND=onnx`. O padrão continua `torch`, e não há detecção nem fallback — backend desconhecido ou grafo ausente derruba o boot (invariante 7). O `torch` saiu das dependências principais para o extra `torch`, e o import dele ficou preguiçoso: `fraus.motor` não o arrasta mais (há teste para isso).

Isso destravou a **medição de RAM que estava pendente**, agora numa venv sem torch instalado (`uv sync --extra dev --extra onnx`), com o app montado por `criar_app_padrao` e 30 conversas atribuídas:

executor	pico de RSS	180 conversas (atribuir)	categorias trocadas
torch	1.848 MB	53,7 s	referência
ONNX, 1 thread (config antiga)	—	45,2 s	0
ONNX, todos os núcleos	1.445 MB	35,0 s	0
ONNX, todos os núcleos, sem arena	1.347 MB	33,2 s	0

Desvio máximo de score contra o torch: `6,5e-5`. A venv sem torch ocupa 430 MB contra 5,0 GB, e a imagem Docker `--build-arg BACKEND=onnx` mede **995 MB**, já com o motor real respondendo `/analisar`.

Duas coisas contrariam a intuição e foram medidas, não supostas:

- 1. A fusão offline -02 do otimizador de transformers saiu mais lenta** (41,2 s contra 35,0 s), embora tenha fundido tudo (12 `Attention`, 24 `SkipLayerNormalization`, 12 `BiasGelu`, GELU exata). A sessão já roda com `ORT_ENABLE_ALL`, que funde na carga; o grafo pré-fundido só perde otimizações que o runtime faria por conta própria. **Recusada.**
- 2. `intra_op_num_threads = 1` era o gargalo.** Fazia sentido quando várias requisições podiam entrar no modelo juntas. Agora o `Motor` deixa uma passada por vez (`FRAUS_INFERENCIAS_SIMULTANEAS`), e a passada usa os núcleos (`FRAUS_ONNX_THREADS`, padrão 0 = o runtime escolhe).

8.6.1 Emoção com int8 só nos pesos — aceita pelo portão, não promovida por padrão

A int8 recusada acima é **dinâmica**: quantiza pesos e computa as ativações em int8. A variante `--precisao int8-pesos` (`MatMulNBits`, 8 bits, blocos de 32, simétrica) guarda os pesos em int8 e mantém a ativação em fp32. Aplicada só à emoção, com satisfação fp32 e ironia int8, nas mesmas 180 conversas:

	ONNX aceito (emoção fp32)	emoção int8-pesos
peso da emoção	417 MB	184 MB
pico de RSS da API	1.347 MB	1.056 MB
180 conversas (atribuir)	33,2 s	49,6 s
desvio máximo de score	0,0001	0,51 ponto
notas / categorias trocadas	0 / 0	0 / 0

Ela passa no portão (zero categorias) e no do script ($\Delta p\ 0,0148 < 0,02$). Mas, ao contrário do fp32, **não é o mesmo número**: meio ponto de score é margem que uma conversa encostada na fronteira 6/7 ou 8/9 pode atravessar num corpus maior que as 180. Por isso ela fica disponível e documentada, **não** como o padrão de `modelos-onnx/`. E 1.056 MB **ainda não cabe** numa máquina de 1 GB, então ela não resolve o caso que justificaria o risco.

A conclusão sobre o encoder compartilhado abaixo não mudou.

8.7 O que sobra como caminho

caminho	tamanho estimado	fidelidade	cabe em 500 MB?
ONNX fp16	$3 \times 208 = 624 \text{ MB} + \sim 50 \text{ MB de runtime}$	quase idêntica	 — mas corta a imagem de 1,75 GB para ~0,67 GB
fp32 na satisfação, int8 no resto	$416 + 106 + 106 = 628 \text{ MB}$	boa no que pontua	
int8 nos três	$318 \text{ MB} + \sim 50 \text{ MB}$	1,7% troca de veredito	

Ou seja: **o orçamento de 500 MB só é alcançável pagando o preço que foi recusado acima**. Isso não é um impasse técnico a resolver com mais tuning — é a Vercel serverless não sendo o alvo certo para este sistema, e vale mais dizer isso do que espremer o número até ele caber.

O ganho continua real fora da Vercel: ONNX fp16 num container corta a imagem em $\sim 2,6\times$ e remove o `torch`, sem custo de fidelidade digno de nota. Esse é o próximo experimento, e ele não precisa de retreino nenhum também.

8.8 Reproduzir

```
uv sync --extra conversao
uv run python scripts/encolher_modelos.py          # converte e afere
uv run python scripts/comparar_backends.py --n 180 # o portão que recusou
```

8.9 Variante experimental: um encoder, três cabeças

O runtime agora aceita `FRAUS_BACKEND=onnx-multitarefa`, mas ele **não é o padrão** e nenhum artefato foi promovido. Diferente da quantização, compartilhar o encoder exige treino: os três checkpoints atuais aprenderam representações diferentes durante o fine-tuning.

O caminho implementado usa destilação. Os modelos atuais são os professores e um BERTimbau único aprende simultaneamente as distribuições de satisfação, emoção e ironia. A saída ONNX possui três tensores nomeados e o `Motor` executa o encoder uma vez por lote — não três vezes com pesos apenas compartilhados.

```
uv sync --extra treino --extra onnx
uv run python scripts/treinar_multitarefa.py corpus.csv --epocas 3
uv run python scripts/comparar_backends.py --multitarefa --n 180
```

O corpus aceita CSV/JSONL com campo `texto` ou TXT com uma frase por linha. Use um corpus público representativo; não exporte conversas de clientes sem base legal e anonimização. O candidato é gravado separadamente e só deve virar `modelos-onnx/bertimbau-multitarefa` depois de cumprir, em conjunto:

- zero troca de categoria nas 180 conversas de regressão;
- métricas por cabeça não inferiores às publicadas nos cartões dos modelos;
- pico de RSS abaixo de 800 MB no container completo;
- inspeção das fatias críticas antes da troca da régua.

O tamanho esperado do grafo fp32 é aproximadamente o de um BERTimbau (cerca de 416 MB) mais três camadas lineares pequenas. O número de RAM continua sendo uma hipótese até o artefato treinado ser medido; o código não o apresenta como ganho concluído.

9. Limitações conhecidas

Esta página existe porque um sistema que estima satisfação **precisa** declarar onde erra. Nada aqui é segredo, e nada aqui está escondido na tela: cada item tem contrapartida visível na interface.

9.1 Limitações operacionais do deploy gratuito

A hospedagem da API deixou de ser pendência em 17/09/2026, mas “publicada” não significa “sem limites”. A função ONNX tem cerca de 1,46 GB e usa Large Functions na Vercel. Depois de esfriar, o primeiro `/saude` observado levou 10,8 s; requisições seguintes foram rápidas, mas não há garantia de latência.

Vercel Hobby, Supabase Free e Oracle Object Storage gratuito oferecem cotas, não SLA. Ao ultrapassá-las, a aplicação pode ser limitada ou pausada. O estado fica no Supabase; o Oracle participa do build, não da inferência. Uma restauração do projeto pode alterar a URI do pooler, e o transaction pooler precisa usar a porta indicada no painel (6543 no deploy atual).

Essas limitações são operacionais. Elas não alteram as limitações científicas abaixo e não autorizam apresentar a estimativa como NPS declarado. Detalhes em [Deploy gratuito na Vercel](#).

9.2 O NPS é inferido, não perguntado

Toda exibição carrega a etiqueta de estimativa e o sublinhado pontilhado de proveniência. Apresentar como NPS declarado seria falso.

O intervalo de confiança cobre a incerteza AMOSTRAL, não a do modelo. Ele responde "quantas conversas sustentam este número"; ele **não** responde "o quanto o modelo pode estar errado" — isso exigiria calibração, que ainda não existe. Um IC apresentado como se cobrisse o erro do modelo é pior que não ter IC nenhum, e por isso a ressalva está impressa junto do número, não só aqui.

Abaixo de **30 atendimentos com sinal** a tela não mostra ponto estimado: mostra pausa e o `n`.

9.3 O sinal de tempo é treinado em dados sintéticos

Nenhum corpus público de review em português tem timestamps de diálogo. O simulador (`fraus/ingest/simulador.py`) gera conversas determinísticas com latências calibradas por literatura de live chat.

E isso tem uma consequência medida, não hipotética. As features de latência não têm teto, e o corpus nunca passou de minutos (medianas de 5/30/200 s). Numa conversa com **3 horas** de espera, medida em 08/09/2026 contra a API real:

```
latencia_primeira_resposta_s -85,405
duracao_total_s             -35,759
latencia_mediana_s          -20,569
latencia_p90_s              -12,326
texto_prob_satisfeito_media  +4,574 ← o texto inteiro
```

O tempo pesa **20x o texto**, e a conversa sai em `4,4e-24`. O peso *por unidade* continua pequeno (0,12-0,16); o que explode é o **z-score** que entra no `StandardScaler`.

O conserto de verdade — `log1p` ou winsorização antes do scaler — exige retreino. Enquanto ele não vem, `GET /saude/deriva` torna o caso **visível**: contra o fusor real, essa conversa dá `|z| = 124,9` em `latencia_mediana_s`, contra um limiar de 4.

9.4 A cabeça de ironia é pouco confiável

Ela reporta acurácia **1.0** no corpus gerado e erra **6 em 10** falas sinceras de atendimento, com 0,999 de confiança — vazamento de marcador de discurso no gerador, já corrigido no gerador e pendente de retreino.

O vazamento não chega mais à nota, porque a ironia saiu do vetor em 04/09/2026 por um motivo diferente e mais grave (ver [Arquitetura](#)). Ela continua sendo exibida por mensagem, com a ressalva colada ao número.

9.5 Desprezo não é uma classe treinada

Nenhum corpus em português a anota. Ela é derivada da díade raiva + nojo (Plutchik, 1980) pela **média geométrica** — que exige as *duas* emoções juntas, enquanto a média aritmética daria meio ponto para raiva pura sem nojo nenhum, o que é raiva, não desprezo.

9.6 Nenhuma feature agregada reverte uma probabilidade saturada

A frase canônica — "*que atendimento maravilhoso, só esperei 3 horas*" — sai promotora sempre que o relógio não a contradiz.

`incongruencia_situacao_negativa` foi criada para alcançá-la e tem peso **-0,193**; ela não vence os **+2,78** de `texto_prob_satisfeito_media` quando o BERTimbau lê a frase como elogio sincero.

Por isso a **contestação** (08/09/2026) **marca em vez de corrigir**: quando `score > 95` e `latencia_mediana_s > 180`, a tela avisa — e o atendimento **continua contando no NPS**, com teste provando isso. Tirá-lo do agregado esvaziaria o indicador em silêncio se o limiar estivesse mal calibrado.

O limiar de score em 95 é o único número do projeto **sem procedência**, e está declarado assim no código e na spec. O de tempo vem de IJHCI 2025.

9.7 O que não dá para consertar sem corpus de atendimento real

Duas coisas ficam condicionadas ao mesmo recurso que não existe publicamente em português — um corpus de **diálogo de atendimento** rotulado:

1. **retreinar a cabeça de ironia** em domínio certo;
2. **a interação tempo × texto** como feature. Ela nasceria com o peso invertido: o texto vem do B2W, a latência sai de distribuição por rótulo, e satisfeito-e-lento está rotulado *satisfeito* por construção. Isso é leitura do corpus, não experimento.

10. Referência de código

10.1 Indicadores

NPS, CSAT, contenção, **falso containment** e a série diária. Todo agregado deste módulo devolve `None` na ausência — nunca `0`, que seria apresentar como medido um número que ninguém mediu.

10.1.1 `fraus.indicadores`

Indicadores agregados.

NPS aqui é INFERIDO do texto, não declarado pelo cliente. A dashboard rotula como estimativa -- apresentar como NPS declarado seria falso.

Faixas canônicas: 0-6 detrator, 7-8 neutro, 9-10 promotor. Categoria SEMPRE derivada no servidor.

`categoria_nps`

```
categoria_nps(score_0_100: float, faixas: dict[Categoria, tuple[int, int]] | None = None) -> Categoria
```

Categoria da nota segundo `faixas` -- por PARAMETRO, nunca global mutável.

Quando a configuração muda as faixas, quem le passa as faixas vigentes; o default é o padrão de fábrica. Assim a faixa continua morando num lugar só e não existe estado global que mude sob os pés de quem já calculou.

`validar_faixas_nps`

```
validar_faixas_nps(faixas: dict[str, tuple[int, int]]) -> None
```

Recusa faixa que não cubra 0..10 de forma contígua, nomeando o problema.

Faixa de NPS com buraco ou sobreposição não é preferência de gosto: seria nota sem categoria (ou com duas), e o erro apareceria muito depois, na leitura de um atendimento qualquer. Levanta `ValueError` -- a borda HTTP traduz para 400 com a mesma frase.

`calcular_nps`

```
calcular_nps(scores: list[float], faixas: dict[Categoria, tuple[int, int]] | None = None) -> float | None
```

Percentual de promotores menos percentual de detratores, em `[-100, 100]`.

Sem score algum devolve `None` -- ausência de dado não é insatisfação, e "NPS +0" seria apresentar como medido um número que ninguém mediu.

`nps_com_intervalo`

```
nps_com_intervalo(scores: list[float], faixas: dict[Categoria, tuple[int, int]] | None = None) -> dict | None
```

NPS inferido com intervalo de confiança de 95% (Wald ajustado).

Hoje "NPS -12" aparece igual com 8 conversas e com 8.000, e a primeira pergunta de quem avalia é "quantas conversas sustentam esse número?". Mostrar ponto estimado sem incerteza é inconsistente com um sistema que já recusa transformar ausência em zero.

A CONTA. O NPS é a média de uma variável em `{-1, 0, +1}` -- promotor +1, detrator -1, neutro 0. Então:

```
media      = p_prom - p_det
variância = (p_prom + p_det) - media**2
```

```
erro = sqrt(variância / n)
ic = media +- 1,96 * erro          (tudo x100 na escala do NPS)
```

O metodo e o Wald AJUSTADO: Wald puro e bootstrap-t sao instaveis em amostra pequena (MDPI Stats, 2026), e o ajuste aqui e a regra de exibicao -- abaixo de `N_MINIMO_NPS` o ponto estimado nao e devolvido, porque com amostra assim ele sugere precisao que nao existe. O `n` VEM PREENCHIDO mesmo assim: a tela precisa dizer quanto falta, nao so que nao sabe.

None quando nao ha score nenhum -- nao existe intervalo de coisa nenhuma.

HONESTIDADE OBRIGATORIA, e ela precisa aparecer na tela junto do numero: este intervalo captura so a incerteza AMOSTRAL. Ele NAO captura a incerteza do MODELO, que exigiria calibracao. Um IC apresentado como se cobrisse o erro do modelo e pior que nao ter IC nenhum.

| calcular_csat

```
calcular_csat(scores: list[float]) -> float | None
```

Percentual de atendimentos com nota ≥ 7 . Sem score algum devolve None.

| mediana

```
mediana(valores: list[float]) -> float | None
```

Mediana, ou None na lista vazia -- nunca 0.0.

Zero seria "respondeu instantaneamente", que e o oposto de "nao da para saber". A serie temporal precisa poder dizer que nao mediu.

| serie_diaria

```
serie_diaria(registros: list[tuple[Conversa, float | None]], faixas: dict[Categoria, tuple[int, int]] | None = None) -> list[dict]
```

NPS inferido e latencia mediana por dia, ordenados do mais antigo.

O dia sai de `iniciada_em` da conversa. A latencia do dia e a mediana das MEDIANAS de cada atendimento, nao a mediana de todas as esperas juntas: um unico atendimento com trinta idas e vindas dominaria o dia inteiro se as esperas fossem jogadas num balde so.

`nps` e `latencia_mediana_s` sao None no dia sem dado, e os dois motivos sao diferentes -- por isso `atendimentos` e `com_score` vem separados. Um dia pode ter atendimento e nenhum score (conversa sem fala do cliente), e apresentar isso como NPS 0 seria inventar medicao.

| containment_rate

```
containment_rate(conversas: list[Conversa]) -> float | None
```

Percentual de conversas resolvidas sem intervencao humana.

None no conjunto VAZIO -- nunca 0.0, que se leria como "nenhum atendimento foi contido", o pior numero da escala, onde nao houve atendimento nenhum.

A distincao que importa e que esta funcao NAO depende de score: conversa sem fala do cliente nao tem NPS nem CSAT e mesmo assim conta como contida (ha teste exigindo 100% nesse caso). O que a ausencia de score nao mede, ela continua medindo. So o conjunto vazio nao tem resposta.

Ate 10/09/2026 devolvia 0.0, e o front nao acreditava: `page.tsx` escrevia `total_conversas ? containment_rate : null`. A guarda no cliente era a prova de que alguem ja tinha sentido o problema -- e ela so protegia a dashboard. Export, webhook e qualquer terceiro lendo /indicadores recebiam o zero. Uma ponta so decide agora.

| contidos_com_score

```
contidos_com_score(registros: list[tuple[Conversa, float | None]]) -> int
```

Quantos atendimentos NAO escalaram e tem score -- o denominador honesto.

A tela precisa dele para dizer "3 de 12" em vez de so um percentual: sem o denominador, 33% sobre tres atendimentos parece a mesma coisa que 33% sobre trezentos.

| falso_containment

```
falso_containment(registros: list[tuple[Conversa, float | None]], faixas: dict[Categoria, tuple[int, int]] | None = None) -> float | None
```

Percentual de atendimentos contidos que sairam DETRATORES.

Conteve e o cliente saiu insatisfeito e sucesso falso: a metrica de contencao sobe enquanto a experiencia piora. So o atendimento COM score entra na conta -- conversa sem fala do cliente nao e nem sucesso nem fracasso, e o denominador precisa dizer sobre quantos se esta falando.

O limiar de insatisfacao e a categoria `detrator` das faixas VIGENTES, recebidas por parametro (invariante 4) -- nunca um 6 digitado aqui.

None quando nenhum atendimento contido tem score -- nunca 0.0, que se leria como "nenhum contido saiu insatisfeito", afirmacao que ninguem mediu.

| tempo_mediano_resposta

```
tempo_mediano_resposta(registros: list[tuple[Conversa, float | None]]) -> float | None
```

Mediana de TODAS as esperas cliente -> resposta do conjunto.

None quando nao ha nenhum par -- nunca zero: zero numa medida de tempo de resposta se le como "respondeu na hora", e conjunto sem par nao mediu espera nenhuma.

| lexico_por_classe

```
lexico_por_classe(registros: list[tuple[Conversa, float | None]], faixas: dict[Categoria, tuple[int, int]] | None = None, limite: int = 6) -> list[dict]
```

Palavras e emojis caracteristicos de cada categoria do conjunto.

Ordenado por DISTINCAO, nao por frequencia: `distincao` e a diferenca entre a fracao do termo nesta classe e a fracao dele nas outras, em [-1, 1]. O termo que aparece em toda parte nao explica classe nenhuma.

Mesma regra da derivacao que a dashboard fazia no cliente (e mantem como plano B): so a fala do CLIENTE conta -- o texto do bot e roteiro --, e conversa sem score fica fora, porque sem categoria nao ha classe onde contar. Emoji e recortado por cluster de grafema, pela mesma extracao do sinal de emoji, para a contagem nao divergir do motor.

10.2 Modelo canônico

`Conversa` e `Mensagem`. Toda fonte de entrada desemboca aqui, e o resto do sistema não sabe de onde o dado veio.

10.2.1 `fraus.modelos`

Modelo canonico de conversa. Toda fonte de dado e normalizada para ca.

`Conversa`

Bases: `BaseModel`

`tem_fala_cliente` property

```
tem_fala_cliente: bool
```

O cliente escreveu alguma coisa -- e o que importa para a OPERACAO.

`tem_sinal_cliente` property

```
tem_sinal_cliente: bool
```

Ha fala do cliente que diga algo sobre satisfacao -- e o que a NOTA exige.

Desde 15/09/2026, conversa em que toda fala do cliente e formula de cortesia ("ok, obrigado", "valeu") nao tem sinal: a mesma formula fecha atendimento bom e ruim. Ver `fraus/cortesia.py`.

`motivo_sem_sinal` property

```
motivo_sem_sinal: Literal['sem_fala_do_cliente', 'so_cortesia'] | None
```

POR QUE nao ha nota -- `None` quando ha sinal.

Existe para a tela nao dizer "o cliente nao falou" a quem escreveu "ok, obrigado", e para ela nao precisar reimplementar a regra da cortesia em TypeScript (invariante 3: regra derivada mora no servidor).

10.3 O fusor e as 39 features

`NOMES_FEATURES` é o **contrato**. O comentário dele conta a história inteira do vetor — por que subiu para 40, por que a ironia saiu, e por que 39 é o número de hoje.

10.3.1 fraus.fusor

Fusor das sete famílias de sinal que entram no vetor (de oito no sistema -- a ironia continua existindo, so nao pontua mais; ver `NOMES_FEATURES`).

LogisticRegression com padronizacao: interpretavel de proposito -- o trabalho precisa defender POR QUE um atendimento recebeu a nota, e coeficiente de regressao logistica responde isso; floresta densa nao.

O peso da latencia e APRENDIDO aqui, nunca arbitrado: a relacao com satisfacao e nao-linear e moderada por contexto.

FusorIncompativelError

Bases: `RuntimeError`

Artefato treinado (`.joblib`) nao bate com `NOMES_FEATURES` vigente.

Distinta de `ModeloAusenteError` (fraus/sinais/texto.py) de proposito: as duas impedem servir predicao (invariante 7), mas o remedio e diferente. Um modelo AUSENTE pede treinar do zero; um modelo INCOMPATIVEL ja existe e foi treinado com um contrato antigo -- o remedio e RETREINAR com o contrato atual, nao gerar um artefato novo do nada. Misturar as duas mensagens levaria quem le o erro a rodar o notebook errado.

Fusor

`escala_do_tempo` property

```
escala_do_tempo: str
```

Escala em que as features de espera foram aprendidas ("segundos" = antiga).

`pontuar`

```
pontuar(features: dict[str, float]) -> float
```

Score 0-100: P(satisfeito) mais P(neutro) pesado por `PESO_NEUTRO_NO_SCORE`.

O score e uma PROJECAO das tres probabilidades num eixo, nao uma quarta predicao: o modelo continua o mesmo, treinado do mesmo jeito. O peso do neutro decide onde a classe do meio pousa nesse eixo -- e, por composicao com a faixa de NPS, em qual categoria ela cai. A tabela e o raciocinio estao na constante.

O que este metodo NAO e: um lugar para consertar predicao. Se o modelo confunde as classes, o conserto e treino -- as medianas por classe do `scripts/medir_faixas.py` sao justamente o que separa os dois casos.

`prever`

```
prever(features: dict[str, float]) -> int
```

Classe predita: 0 insatisfeito, 1 neutro ou 2 satisfeito.

`contribuicoes`

```
contribuicoes(features: dict[str, float]) -> dict[str, float]
```

Quanto cada feature empurrou a nota DESTA conversa, com sinal.

Eixo: coeficiente da classe satisfeito menos coeficiente da classe insatisfeito, multiplicado pelo valor JA PADRONIZADO da feature (o `StandardScaler` do pipeline aplicado, nunca o valor bruto). Positivo empurrou a nota para cima (rumo a satisfeito); negativo puxou para baixo (rumo a insatisfeito). E o numero da CONVERSA, diferente de `importancias`, que e o peso medio GLOBAL aprendido pelo modelo.

Se o modelo aprendeu menos de tres classes e nao tem as duas pontas (insatisfeito e satisfeito) para formar a diferenca, nao ha eixo interpretavel: a contribuicao volta zerada para todas as features, em vez de estourar.

| eixo_global

```
eixo_global() -> dict[str, float]
```

Quanto cada feature empurra a nota NO MODELO INTEIRO, com sinal.

Diferente de `importancias`, que e o peso ABSOLUTO (diz que a feature pesa, nao para que lado), e de `contribuicoes`, que e o numero de UMA conversa. O grafo da memoria consome este: ele responde "o que o modelo aprendeu que caracteriza um detrator".

Nao ha padronizacao aqui porque nao ha conversa: e o coeficiente cru.

Fusor nao treinado devolve `{}` -- e a ausencia de eixo, que o grafo distingue de "todas as features pesam zero".

| distribuicao_de_treino

```
distribuicao_de_treino() -> dict[str, dict[str, float]]
```

Media e desvio de cada feature NO CORPUS DE TREINO, por feature.

O `StandardScaler` treinado guarda `mean_` e `scale_` -- e isso e o retrato da distribuicao em que o modelo aprendeu, ja gravado dentro do artefato. LER NAO E TREINAR: nao precisa do corpus, nem de notebook, nem de GPU.

Existe para responder uma pergunta que a invariante 10 so sabia fazer no notebook: "esta conversa esta dentro da faixa em que o modelo foi treinado?". A guarda de vazamento olha a distribuicao no CORPUS; em runtime aparece latencia de tres horas que o corpus nunca teve, o z-score explode e o relógio assume a nota.

Fusor nao treinado devolve `{}` -- ausencia do retrato, que e diferente de "todas as medias sao zero".

| z_das_features

```
z_das_features(features: dict[str, float]) -> dict[str, float]
```

Quantos desvios de treino cada feature desta conversa esta da media.

E exatamente o que o `StandardScaler` entrega ao modelo -- o mesmo numero que multiplica o coeficiente. Por isso a sonda usa `transform` em vez de refazer a conta: uma segunda implementacao da padronizacao divergiria da real no dia em que o pipeline mudasse, e a rota de diagnostico passaria a diagnosticar a si mesma.

Feature constante no treino tem `scale_` = 1 por decisao do proprio scikit-learn, entao nao ha divisao por zero aqui.

Fusor nao treinado devolve `{}`.

| assinatura

```
assinatura() -> str | None
```

Impressao digital dos pesos aprendidos -- muda a cada retreino.

Hash do que DECIDE a nota: media e escala do scaler, coeficientes, interceptos, ordem das classes e a escala do tempo. Existe para o banco saber com qual modelo cada conversa foi pontuada (`Motor.regua`). Fusor nao treinado nao tem assinatura.

importancias

```
importancias() -> dict[str, float]
```

Peso absoluto medio de cada feature -- alimenta a explicacao na dashboard.

carregar classmethod

```
carregar(caminho: Path) -> Fusor
```

Carrega o artefato e VALIDA a forma contra `NOMES_FEATURES` antes de devolver.

`joblib.load` puro (o comportamento anterior) nao checava nada: um artefato de 40 features carregava sem erro contra um contrato de 38, a API subia normalmente, `/modelo/simular` funcionava (nao passa pelo fusor) e enganava quem testava a mao -- o `ValueError` do `StandardScaler` so estourava como HTTP 500 na primeira pontuacao real, em `/ingestao` ou `/conversas/importar`. Isso confundiu duas vezes na mesma semana. A invariante 7 pede falha alta e EXPLICITA para modelo incompativel, e "explicita" nao basta se so acontece na primeira predicao -- precisa ser na CARGA, que e quando a API sobe.

montar_features

```
montar_features(conversa: Conversa, classificador, classificador_emocao, curadoria=None) -> dict[str, float]
```

Junta as SETE familias de sinal do vetor numa linha unica de features.

Sao sete familias NO VETOR -- oito sinais existem no sistema, porque a ironia continua sendo lida por mensagem (ver `Motor._ironia_de`), so nao entra aqui. Ver o comentario acima de `NOMES_FEATURES` para o porque.

Os dois classificadores (texto e emocao) sao OBRIGATORIOS desde que o contrato subiu para 35: emocao deixou de ser leitura decorativa e passou a mover a nota. Aceitar `None` aqui produziria vetor incompleto, e vetor incompleto vira `KeyError` la em `vetorizar` -- com a diferenca de que o erro apontaria para o lugar errado. Note que NAO ha `classificador_ironia` aqui: a cabeca de ironia continua obrigatoria para o Motor (invariante 7), mas este montador de vetor nao precisa mais dela.

`curadoria` e opcional e chega POR PARAMETRO, nunca por estado global: e o que o analista ensinou ao lexico, lido a cada requisicao. Ela alcanca TRES familias -- `lexico_*`, `emoji_*` e `incongruencia_*`, que le os dois lexicos por dentro e portanto herda a curadoria deles -- e MAIS NENHUMA: o contrato continua de 39 chaves (invariante 9), e o que ela muda e o VALOR dessas familias, jamais o conjunto de features.

vetorizar

```
vetorizar(features: dict[str, float]) -> list[float]
```

Ordem canonica. Feature faltando e `KeyError`; feature sobrando e `ValueError`.

Nenhum dos dois casos pode virar zero silencioso (invariante 9): falta ja estourava; sobra nao estourava -- um dict com as 39 chaves certas MAIS uma extra passava batido, gerando um vetor do tamanho certo por acaso. E justamente o cenario que aconteceria se uma familia de sinal saísse do contrato (como `ironia_*` saiu em 04/09/2026) mas continuasse sendo produzida por engano em algum chamador: o vetor ficaria certo e a regressao passaria despercebida.

10.4 Os sinais

As oito famílias que leem a conversa. Todas leem **apenas a fala do cliente** (`mensagens_cliente`) — e isso não é detalhe de implementação, é a fronteira que mantém o sistema fora da proibição de reconhecimento de emoção no local de trabalho. Ver [Conformidade](#).

10.4.1 Texto — o único que pontua sozinho

10.4.2 `fraus.sinais.texto`

Sinal de texto: BERTimbau fine-tunado, rodando em CPU.

Predicao POR MENSAGEM, nunca por conversa inteira -- e isso que permite a atribuicao por sentenca na dashboard ("quais trechos puxaram a nota").

BERTimbau supera as variantes multilingues em classificacao de sentimento PT-BR (Souza, Nogueira, Lotufo -- arXiv:2201.03382).

`ModeloAusenteError`

Bases: `RuntimeError`

Modelo nao encontrado ou ilegivel. Falha alta: sem modelo nao ha predicao.

`ClassificadorTexto`

`prever_mensagens`

```
prever_mensagens(textos: list[str]) -> list[list[float]]
```

Probabilidades [insatisfeito, neutro, satisfeito] para cada texto.

`carregar_torch`

```
carregar_torch(caminho_modelo: Path, rotulo: str)
```

Tokenizador e modelo do checkpoint, com o torch importado SO AQUI.

O torch (497 MB) deixou de ser dependencia principal: com `FRAUS_BACKEND=onnx` a API pontua sem ele. Importar no topo do modulo faria `fraus.motor` -- que todo mundo importa -- exigir torch mesmo quando nenhum checkpoint torch vai ser carregado. Faltar o torch quando o backend E torch continua falha alta (invariante 7), com o remedio escrito.

`prever_torch`

```
prever_torch(tokenizador, modelo, textos: list[str]) -> list[list[float]]
```

Softmax por texto, num lote com padding pela mensagem mais longa.

10.4.3 Emoção

10.4.4 `fraus.sinais.emocao`

Sinal de emocao: segundo BERTimbau fine-tunado, rodando em CPU.

Requisito de banca: distinguir as SETE emocoes humanas. A cabeca treinada tem sete classes -- as seis basicas de Ekman (1992) mais o neutro, que nao e emocao e sim a ausencia dela. A setima emocao, o DESPREZO, nao e treinada: nenhum corpus rotulado

em português a anota, e treinar só ela em corpus de outra procedência ensinaria o modelo a reconhecer o estilo do texto em vez do desprezo. Ela é derivada da diade primária raiva + nojo, que é como o Plutchik (1980) a define -- ver `desprezo_derivado`.

Corpus: `go_emotions_ptbr`, tradução PT-BR do GoEmotions (Demszky et al., 2020: 58k comentários do Reddit, 27 emoções + neutro), reduzido às seis de Ekman pelo mapeamento oficial que o próprio GoEmotions publica.

LIMITAÇÃO METODOLÓGICA, a declarar no relatório: o corpus foi traduzido por máquina, sem revisão humana. Por isso a avaliação reporta também o XED-pt, que não passou por tradução automática. Ver docs/treinamento.md.

ClassificadorEmocao

`prever_mensagens`

```
prever_mensagens(textos: list[str]) -> list[list[float]]
```

Probabilidades das sete classes, na ordem de Nomes_Emocoes.

desprezo_derivado

```
desprezo_derivado(prob_raiva: float, prob_nojo: float) -> float
```

Desprezo como diade primária raiva + nojo (Plutchik, 1980).

Média GEOMÉTRICA, não aritmética: desprezo exige as DUAS emoções juntas, e a média aritmética daria 0.5 para raiva pura sem nojo nenhum -- o que é raiva, não desprezo. O produto puro subestimaria (0.7 e 0.7 viram 0.49); a geométrica devolve 0.7, que é a leitura certa de "as duas altas".

features_emocao

```
features_emocao(conversa: Conversa, classificador) -> dict[str, float]
```

Média de cada emoção nas falas do cliente, mais o desprezo derivado.

São OITO features: as sete classes treinadas e o desprezo da diade. A média é sobre as mensagens do cliente, coerente com o sinal de texto -- fala do bot não carrega emoção do cliente.

10.4.5 Tempo

10.4.6 fraus.sinais.tempo

Sinal de tempo.

Latência entra como FEATURE aprendida, nunca como penalidade linear: o efeito sobre satisfação é não-linear (CSAT pico entre 5-10s, queda de 2-3 pontos por minuto extra, 57% de abandono acima de 3min) e moderado por contexto -- indicador de digitação e suporte emocional atenuam o dano (From Seconds to Sentiments, IJHCI 2025).

Nada aqui é persistente: tudo deriva dos timestamps do modelo canônico.

latencias_da_conversa

```
latencias_da_conversa(conversa: Conversa) -> list[float]
```

Intervalo de cada mensagem do cliente até a próxima resposta.

PÚBLICA de propósito: a série temporal (`fraus.indicadores.serie_diaria`) e a transcrição da dashboard precisam da MESMA regra que vira feature do modelo. Duas definições de latência -- uma para treinar, outra para exibir -- fariam o gráfico contar uma história que o usuário nunca viu.

10.4.7 Léxico

10.4.8 fraus.sinais.lexico

Sinal lexico: SentiLex-PT02 com escopo de negacao.

Fonte: SentiLex-PT02 -- Silva, Carvalho e Sarmento, "Building a Sentiment Lexicon for Social Judgement Mining" (PROPOR 2012), CC-BY. 79.189 formas flexionadas depois da conversao, em `fraus/dados/sentilex_pt02.csv`. O `scripts/preparar_sentilex.py` documenta o que foi normalizado.

POR QUE um lexicon se o BERTimbau ja le o texto: porque o lexicon nao foi treinado nos mesmos dados nem carrega os mesmos vieses. Dois sinais de procedencia independente que concordam sustentam a nota melhor que um sozinho, e discordancia entre eles e informacao -- e o formato tipico da IRONIA, texto lexicamente positivo com contexto negativo.

A NEGACAO e tratada por escopo, nao por saco de palavras: "nao foi otimo" inverte a polaridade de "otimo". Sem isso o lexicon leria a frase como positiva, que e o erro classico de analise lexical de sentimento.

LIMITACAO DO RECURSO, a declarar no relatorio: o SentiLex e um lexicon de JULGAMENTO SOCIAL -- anota polaridade dirigida a entidades humanas. Ele e forte no adjetivo que julga ("pessimo", "otimo", "incompetente") e NEUTRO em verbo de afeto do proprio falante: `gostar`, `adorar` e `odiar` valem 0 nele. Por isso o sinal lexico COMPLEMENTA o BERTimbau e nao o substitui -- o transformer e quem le "adorei o produto".

`sem_acento`

```
sem_acento(texto: str) -> str
```

Remove diacriticos, preservando o resto.

O cliente de chat nem sempre acentua, entao `otimo` precisa achar `ótimo`. Quem consome constroi um INDICE com isto -- nunca compara termo a termo em laco, que seria varrer o lexicon inteiro a cada token.

`polaridade_do_termo`

```
polaridade_do_termo(termo: str, curadoria: Curadoria | None = None) -> int
```

Polaridade em -1/0/1. Termo fora do lexicon e nao curado vale 0.

`anotar_texto`

```
anotar_texto(texto: str, curadoria: Curadoria | None = None) -> list[tuple[str, int, bool]]
```

Termos do lexicon achados em `texto`: (termo, polaridade final, negado).

A polaridade final ja vem com a negacao aplicada. `negado` diz se ela foi invertida -- e o que alimenta a feature de negacao, e o que permite mostrar na dashboard POR QUE um termo positivo contou como negativo.

`features_lexico`

```
features_lexico(conversa: Conversa, curadoria: Curadoria | None = None) -> dict[str, float]
```

Tres features do lexicon sobre as falas do cliente.

`lexico_cobertura` existe para o fusor saber QUANTA evidencia lexical sustenta a polaridade media: media de -1 apoiada em um termo vale menos que a mesma media apoiada em dez.

10.5 Evidência e deriva

Os dois módulos que respondem **"dá para confiar nesta medida?"** — um por conversa, outro pelo modelo inteiro. Nenhum dos dois altera predição: são diagnóstico.

10.5.1 Evidência fraca — a cabeça tracejada

10.5.2 frauds.evidencia

Evidencia fraca: tem dado, e não há material suficiente para afirmar.

A interface tinha DUAS formas -- cabeça cheia (medido) e cabeça vazada (não medido, `CabecaVazada.tsx`). Faltava a terceira, e o buraco era visível: uma conversa cujo cliente escreveu só "ok" saía CHEIA, com a mesma confiança visual de uma conversa de 40 turnos.

```
cheia      medido, com evidencia
tracejada  medido, evidencia fraca
vazada     não medido
```

O QUE ESTE MÓDULO NÃO FAZ, e é a decisão mais importante dele: não usa a probabilidade do modelo. Probabilidade não calibrada NÃO É CONFIANÇA, e apresenta-la como se fosse seria a desonestidade exata que este projeto recusa em todo lugar -- calibrar exige retreino, que está fora de escopo. A medida aqui é EVIDÊNCIA OBSERVÁVEL: quantas vezes o cliente falou e quanto ele escreveu. Auditável por quem abrir o transcript, e defensável sem apelar para propriedade nenhuma do modelo.

Pelo mesmo motivo o campo se chama `evidencia_fraca`, e não `confianca`: o segundo nome prometeria uma calibração que não existe.

motivos_de_evidencia_fraca

```
motivos_de_evidencia_fraca(conversa: Conversa, palavras_minimas: int = PALAVRAS_MINIMAS) -> list[str]
```

Por que a evidência desta conversa é fraca -- em texto, não em código.

"Evidência fraca" sozinho não aciona ninguém; "uma única mensagem do cliente" aciona. A tela imprime estes motivos ao lado da forma, porque forma sozinho obriga o leitor a conhecer a convenção -- a mesma regra que faz `CabecaVazada` exigir rótulo textual.

Lista vazia = evidência suficiente.

evidencia_fraca

```
evidencia_fraca(conversa: Conversa, palavras_minimas: int = PALAVRAS_MINIMAS) -> bool | None
```

A conversa tem score, mas pouca fala para sustenta-lo?

`None` sem fala nenhuma do cliente -- e o estado "sem sinal", que já tem forma própria. "Sem sinal" e "sinal fraco" são coisas DIFERENTES, e confundi-las na interface perderia a distinção que o produto inteiro defende: ausência não é um valor baixo.

10.5.3 Deriva de distribuição — a invariante 10 em runtime

10.5.4 frauds.deriva

Deriva de distribuição: a invariante 10 como alarme de runtime.

A invariante 10 diz que o corpus de treino não pode entregar o rótulo, e o critério de `docs/treinamento.md` é explícito: "se as features que lideram forem as circunstanciais (tempo, contagem de turnos), procure o vazamento". Essa guarda roda no NOTEBOOK, sobre o corpus.

O que ela não alcança: o corpus do sinal de tempo nunca passou de minutos (medianas de 5/30/200 s no simulador), e em produção aparece conversa com três horas de espera. Medido em 08/09/2026 contra a API real, uma conversa assim recebe `latencia_primeira_resposta_s -85,4` contra `texto_prob_satisfeito_media +4,6` -- o tempo pesa 20x o texto e o score sai em 4,4e-24. O peso POR UNIDADE continua pequeno (0,12-0,16); o que explode e o z-score que entra no scaler.

Este módulo não corrige nada disso. O conserto de verdade -- `logit` ou winsorização antes do scaler -- exige RETREINO, e retreino está fora de escopo. O que ele faz é tornar o caso VISÍVEL: dizer quais features da amostra recente estão fora da faixa em que o modelo aprendeu, com o nome de cada uma.

Por que não fazer sem retreinar: o `StandardScaler` treinado já carrega `mean_` e `scale_` por feature dentro de `modelos/fusor.joblib`. Ler não é treinar.

A parte PURA mora aqui, separada da rota, porque ela não precisa de BERTimbau nenhum -- e o que permite a regra falhar em milissegundos de pytest.

| `resumo_de_deriva`

```
resumo_de_deriva(zs_por_conversa: list[dict[str, float]], distribuicao: dict[str, dict[str, float]], limiar: float = LIM_IAR_Z) -> dict | None
```

Quais features da amostra recente saíram da faixa de treino, e quanto.

`zs_por_conversa` é uma lista de `{feature: z}` -- o valor JA padronizado pelo scaler do fusor, que é literalmente o número que multiplica o coeficiente na predição.

O z reportado por feature é o **pior módulo da amostra**, nunca a média: uma conversa fora da faixa entre cem dentro ainda é o caso a investigar, e a média a diluiria até virar ruído de arredondamento -- apagando exatamente o evento que esta rota existe para achar.

Amostra vazia devolve `z_absoluto_observado: None` e `dentro_da_faixa: None`, nunca `0.0 / True`: zero ali se leria como "exatamente na média do treino", que é a afirmação mais tranquilizadora possível a respeito de uma amostra que não existe.

`None` quando o fusor não tem retrato de treino (não treinado): ausência de diagnóstico, diferente de um relatório vazio com cara de saudável.

NAO ALTERA PREDIÇÃO NENHUMA. É diagnóstico.